

基于预训练词向量的中文影评情感分类模型比较研究

《人工智能导论》实验二课程报告

课程：人工智能导论

实验主题：CNN 与 RNN 在情感分类任务中的应用与比较

报告文件名：2023013015-许晴川.pdf

摘要

本文围绕中文影评情感二分类任务，对 MLP、TextCNN、BiRNN、BiLSTM 与 BiGRU 五个主模型进行了实现、训练与比较，并额外纳入轻量 Transformer 编码器作为扩展对照。实验基于已完成分词的中文影评数据集展开，训练集、验证集与测试集规模分别为 19,998、5,629 与 369 条，并使用 50 维中文预训练 Word2Vec 词向量作为初始输入表示。本文首先说明数据处理、词表构建、向量初始化、模型训练与验证集早停等实验流程；随后通过结构图、Accuracy/F1 指标曲线、结果表格和参数对比实验，对不同结构的性能、收敛特征与鲁棒性进行系统比较。结果表明，课程要求覆盖的 CNN/RNN/MLP 主实验模型均满足 80% 以上准确率指标，其中 TextCNN 在测试集上取得最优表现，Accuracy 为 0.8564，F1 为 0.8490。在主实验之外，本文还追加了“BiRNN→BiLSTM→BiLSTM+Attention”与“单卷积核 CNN→TextCNN→数据增强”两条改进路线；结果显示，BiLSTM 相比 BiRNN 的测试集 F1 提升约 0.030，而多卷积核 TextCNN 相比单卷积核 CNN 的测试集 F1 提升约 0.035，说明结构升级比轻量文本增强更稳定。进一步的超参数与跨域测试表明，门控循环结构在域内任务上整体较稳定，但当测试文本迁移到微博语体时，所有模型性能均明显下降，说明当前方法对领域分布偏移的适应能力仍然有限。

关键词：情感分类；预训练词向量；TextCNN；循环神经网络；Transformer；模型比较

1 模型结构图

本节按照课程评分标准，集中展示 baseline、CNN 与双向循环模型的结构图，并结合具体样本说明各组成部分在前向传播和训练阶段的作用。为了便于横向比较，所有模型统一以“输入准备 - 特征提取 - 分类输出 - 参数更新”为主线进行说明。

本节直接给出课程作业要求的模型结构流程图，并按“输入准备 -> 特征提取 -> 分类输出 -> 训练更新”的顺序解释每个组成部分。下面统一用样本“1 你好”举例：其中 1 是真实标签，表示正类；真正进入模型计算的文本是“你好”，若它被分词成一个 token，则长度向量为 1，后续不足 80 的位置都用 PAD=0 补齐。

1.1 MLP 结构说明



图1 MLP 情感分类模型流程图

这张图对应的是“先做 masked mean pooling，再做浅层分类”的 baseline。它的重点不是保留词序，而是把有效 token 的词向量平均成句向量，然后交给两层感知机做二分类，因此结构最简单、训练速度也最快。

对应编号	模块	功能说明（统一以样本“1 你好”为例）
1	分词文本样本	原始样本“1 你好”里的1是真实标签，表示正类；真正送入模型计算的文本只有“你好”。如果分词结果把“你好”视为一个 token，那么当前样本的有效 token 序列就是 [你好]。

2-3	ID 序列 + 长度向量	词表先把“你好”映射成 id(你好)，再按最大长度 80 补齐为 [id(你好), 0, 0, ..., 0]；同时记录真实长度 lengths=1。这里的 0 是 PAD，用来补齐 batch 内不同长度的句子。
4	词嵌入层	Embedding 把离散编号变成 50 维连续向量，于是第 1 个位置得到 e(你好)，后面 79 个 PAD 位置都对应 padding 向量 0，整个张量尺寸变成 [B,80,50]。
5-8	Mask -> 求和 -> 归一化 -> 平均句向量	mask 只保留真实 token，对本样本就是 [1,0,0,...,0]。masked sum 之后只剩下 e(你好)；再除以真实长度 1，就得到平均池化后的句向量。若句子更长，这一步就是把所有有效词向量做 masked mean pooling。
9-12	Dropout -> Linear(50,128) -> ReLU -> Dropout	第一层 dropout 只在训练时生效，用来随机屏蔽部分维度，减少过拟合；Linear(50,128) 把 50 维句向量投影到 128 维隐藏空间；ReLU 保留有效正激活；第二个 dropout 继续约束分类头不要过度依赖少数特征。
13-15	输出层 -> Logits -> Argmax	Linear(128,2) 输出两个类别分数 [z0,z1]。这两个值是 logits，不是概率；如果 z1 > z0，argmax 就会把样本预测成标签 1。若需要概率解释，还要再经过 softmax。
训练分支	CrossEntropyLoss -> 反向传播 -> AdamW	训练时把 logits 和真实标签 1 一起送入交叉熵损失；若模型已经把“你好”判得更偏向正类，损失就更小。随后反向传播把误差传回 Embedding 和两层全连接层，AdamW 负责更新参数。

1.2 TextCNN 结构说明

TextCNN 情感分类模型流程图

输入输出统一记号: B=batch size, L=80, E=50, C=2



图2 TextCNN 情感分类模型流程图

TextCNN 的关键在于“多尺度局部模式抽取”。它并行使用 3、4、5 三种卷积核，分别去抓 3-gram、4-gram、5-gram 级别的情感触发短语，再把三路最强响应拼成句向量，因此对“非常喜欢”“一点也不好”这类局部搭配尤其敏感。

表1 TextCNN 各部分功能与“1 你好”示例

对应编号	模块	功能说明（统一以样本“1 你好”为例）
1-4	样本解析 -> ID -> 长度 -> Embedding	“1 你好”会先拆成标签 1 和文本“你好”。文本部分映射为 [id(你好), 0, ..., 0]，真实长度为 1，再被 Embedding 映射成 [B,80,50] 的词向量序列；此时只有第一个时间步是真实词向量，其余位置都是 PAD 向量。
5	Unsqueeze	TextCNN 的卷积层使用 Conv2d，因此需要把嵌入张量从 [B,80,50] 扩成 [B,1,80,50]。新增的这一维可以理解为“单通道文本图像”的通道维。
6	三路并行卷积模块	三组卷积核尺寸分别是 (3,50)、(4,50)、(5,50)，每组 128 个 filter。它们沿时间轴滑动，一次覆盖 3、4、5 个相邻 token，并把整条 50 维词向量一起看进去。对“你好”这种短句来说，窗口里会同时看到真实词和 PAD，模型仍可学习“句首出现正向词”这一局部模式。
6A-6C	ReLU + 时间维 Max Pooling	每一路卷积输出都会先过 ReLU，保留有效激活，再在时间维做最大池化。最大池化的含义是：不管某个情感模式出现在句子哪个位置，只保留这一路 filter 在整句上的最强响应值，因此每一路最终压缩成 128 维向量。

7	Concat	三路池化结果分别是 [B,128]，拼接后得到 [B,384]。这一步把 3-gram、4-gram、5-gram 三种局部情感证据放到同一个句向量里，让分类器同时参考不同长度的短语模式。
8-10	Dropout -> 全连接分类层 -> Logits	Dropout 在训练时随机丢弃部分卷积特征，防止模型只记住训练集里的固定搭配；Linear(384,2) 再把拼接后的局部模式映射成两个类别分数。输出的 [z0,z1] 仍然是 logits，而不是概率。
12	最终预测	如果 z1 大于 z0，argmax 预测为标签 1；否则预测为标签 0。对样本“1 你好”来说，理想情况是与“你好”相关的正向卷积核响应更强，使 z1 高于 z0。
训练分支	损失与参数更新	训练时同样使用 CrossEntropyLoss 比较 logits 与真实标签 1，再通过反向传播同时更新词向量、三路卷积核和输出层参数。也就是说，TextCNN 学到的不是词频统计，而是“哪些局部 n-gram 模式最能区分正负情感”。

1.3 BiRNN / BiLSTM / BiGRU 结构说明

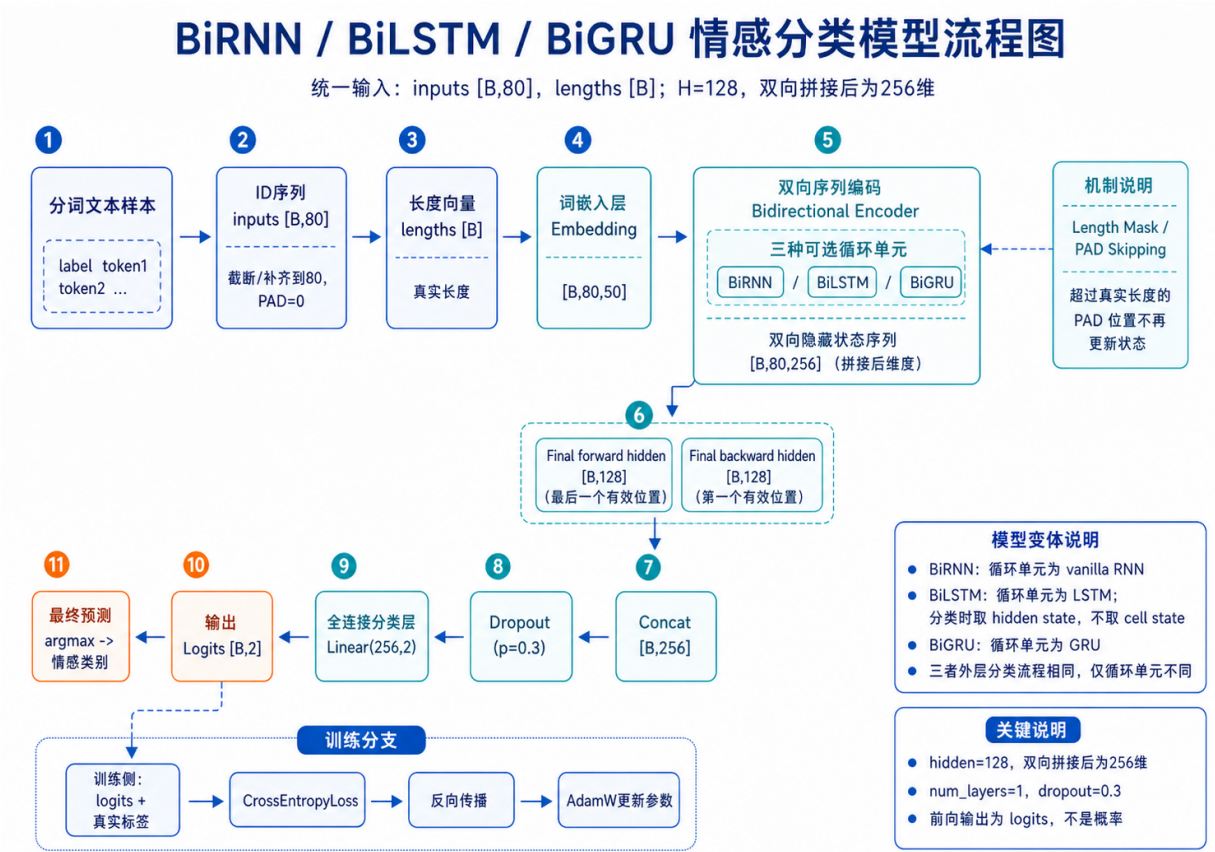


图3 BiRNN / BiLSTM / BiGRU 情感分类模型流程图

这三种循环模型共用同一套外层流程：先做词嵌入，再由双向编码器读取上下文，最后拼接前向和后向状态做分类。它们真正的差别只在循环单元内部的信息更新方式，因此适合放在一张总图里对照说明。

表2 双向循环模型各部分功能与“1 你好”示例

对应编号	模块	功能说明（统一以样本“1 你好”为例）
------	----	---------------------

1-4	样本解析 -> ID -> 长度 -> Embedding	“1 你好”同样会被处理成 [id(你好), 0, ..., 0]，真实长度 lengths=1，嵌入后得到 [B,80,50]。这一段和 MLP、TextCNN 一样，负责把离散文本变成后续编码器可处理的连续向量序列。
5	双向序列编码	双向编码器会从左到右和从右到左各跑一遍。对这个样本来说，只有第一个位置是真实 token，所以前向和后向状态都只会在“你好”这一位置更新一次；后面 79 个 PAD 位置会被长度掩码跳过，不再改写隐藏状态。
6	最终前向/后向隐藏状态	前向隐藏状态表示“从句首读到最后一个有效 token 后”的摘要，后向隐藏状态表示“从句尾反向读到第一个有效 token 后”的摘要。对长度为 1 的样本，两者都围绕“你好”本身形成表征；对长句，两者则分别保留左右文信息。
7-9	Concat -> Dropout -> Linear(256,2)	模型把前向 128 维和后向 128 维状态拼成 256 维句向量，再经过 dropout 和线性层输出两个类别分数。若与“你好”相关的正向上下文特征更强，最终就会得到 $z1 > z0$ 。
10-11	Logits -> 最终预测	线性层输出的是 logits $[z0, z1]$ ，argmax 才是最终类别。训练时仍然通过交叉熵把预测结果拉向真实标签 1。
模型差异	BiRNN / BiLSTM / BiGRU 的不同点	三者的外层流程完全一致，差别只在第 5 步循环单元内部。BiRNN 使用最基础的 tanh 状态更新，结构最简单但长距离记忆最弱；BiLSTM 额外维护 cell state，并通过输入门、遗忘门、输出门控制信息流，更适合保留长期依赖；BiGRU 用更新门和重置门做较轻量的门控，在参数量和效果之间取得折中。
训练分支	损失与参数更新	训练阶段把 logits 与标签 1 送入 CrossEntropyLoss，误差会反向传播到双向循环单元和词向量。由于循环网络按时间步更新状态，它比 MLP 和 CNN 更强调“前后文是如何逐步传递过来的”。

综合来看，三类结构的主要区别不在“有没有用到词向量”，而在“怎样把词向量组织成句向量”：MLP 做平均，最省计算但不保留词序；TextCNN 抓局部 n-gram，擅长识别短语级情感模式；双向循环模型按时间步传递状态，最能体现上下文顺序与长期依赖。训练阶段它们都统一输出 logits，并通过交叉熵损失、反向传播与 AdamW 优化器完成参数更新。

2 实验流程描述

2.1 任务背景

情感分析旨在识别文本所表达的主观态度与情绪倾向，是自然语言处理中的典型文本分类任务。在中文互联网环境中，影评、微博和商品评论等文本往往具有较强的主观性，因此情感分类既具有现实应用价值，也能够作为检验文本表示与神经网络建模能力的基础问题。

本实验要求基于预训练词向量实现 CNN 与 RNN 模型，并在中文情感分类任务上比较其效果。为使横向比较更完整，本文进一步实现了 MLP 基线，并额外补充轻量 Transformer 编码器作为扩展对照，在相同数据划分和评价指标下分析不同结构对局部

模式提取、序列依赖建模和训练稳定性的影响。

2.2 数据加载与预处理

实验数据包含 `data/train.txt`、`data/validation.txt`、`data/test.txt` 三个文本文件，以及 50 维预训练词向量 `data/wiki\_word2vec\_50.bin`。每条样本由二分类标签和已经分词的中文句子构成。训练集、验证集与测试集规模分别为 19,998、5,629 和 369 条，预训练词向量对训练语料 token 的覆盖率为 97.18%。

在具体实现上，程序会先逐行读取三份文本文件，并把每一行解析成“标签 + 分词结果”两部分：第一列转成 `0/1` 标签，后续所有词保留为原始 token 列表。随后，程序只基于训练集统计词频并构建词表，而不会把验证集和测试集一起并入词表生成过程；这样做是为了避免信息泄漏。构建词表时，最前面固定加入“与”两个特殊标记，分别用于补齐长度和承接未登录词。

词表确定后，程序并不会把整个 Word2Vec 文件完整加载进内存，而是只抽取当前词表中真正会用到的词向量。对于命中预训练词向量的词，直接使用对应 50 维向量初始化 embedding；对于没有命中的词，则按照已命中向量的均值和标准差做随机初始化，以保证尺度一致；“向量始终强制置零，防止补齐位置在池化、卷积和循环状态传播时引入伪信息。这样处理后，词向量矩阵既保留了预训练语义，又能兼容训练语料中未命中的词。

接下来，程序把每条句子统一截断或补齐到长度 80，并同时保存该句子的真实长度 `lengths`。因此，每个数据切分最后都会被编码成三组张量：`inputs` 表示词编号矩阵，`lengths` 表示真实长度，`labels` 表示情感标签。固定长度张量便于后续按 batch 训练，而真实长度又保证模型能够区分“真实 token”和“补齐位置”。为了提高重复运行时的效率，程序还会把预处理结果缓存到磁盘；如果 `max\_len`、`min\_freq` 和随机种子配置没有变化，后续运行时就直接复用缓存，而不重新构建词表和抽取词向量。

2.3 模型训练与结果评估

在模型训练阶段，主实验分别构建 MLP、TextCNN、BiRNN、BiLSTM 与 BiGRU，并在统一数据划分下进行训练；Transformer 也沿用相同数据流程单独训练，但在报告中作为扩展对照单独分析。优化器统一采用 AdamW；训练阶段使用带 label smoothing 的交叉熵损失，验证和测试阶段则改用普通交叉熵单独统计指标。循环模型额外设置更小的词向量学习率、短暂 warmup 和更保

守的早停轮数，以减少手写循环单元在训练初期的不稳定现象。这样的设置既保证了实验可重复性，也更便于公平比较不同结构之间的收敛特征。

更具体地说，每个模型训练前都会先根据对应的 `'batch_size'` 构造训练、验证和测试三个 `DataLoader`，其中只有训练集开启随机打乱，验证集和测试集保持固定顺序。进入每一轮训练后，程序会依次完成以下步骤：从 `DataLoader` 中取一个 batch；把 `'inputs'`、`'lengths'` 和 `'labels'` 移动到当前设备；前向计算得到 logits；根据标签计算损失；执行反向传播；若配置中启用了 `'grad_clip'`，则在参数更新前先做梯度裁剪；最后调用优化器完成一次参数更新。对于循环模型，这一步尤其重要，因为循环结构在较长序列上更容易出现梯度爆炸。Transformer 相关实验沿用同一数据预处理流程，只是在特征提取阶段改用自注意力编码器。每完成一轮训练，程序都会立即在验证集上关闭梯度进行完整评估，统一计算验证损失、Accuracy、Precision、Recall 和 F1，并把这些信息连同本轮学习率、耗时和词向量是否解冻一起记录进 history。随后，程序以验证集 F1 为第一优先级、验证集 Accuracy 为第二优先级判断当前轮是否优于历史最佳；如果更优，就立即保存当前模型参数为 checkpoint，并把该轮记为最佳轮次。若连续若干轮没有提升，且已经达到最小训练轮数要求，就触发 early stopping，提前结束训练，从而减少无效训练和过拟合风险。

训练结束后，程序不会直接用“最后一轮”的参数汇报结果，而是先把模型权重回滚到验证集表现最好的那一轮，再分别在验证集和测试集上重新评估一次，得到最终写入 `'*_metrics.json'` 的指标结果。与此同时，最佳模型参数会保存成 `'*_best.pt'`，主实验的所有结果进一步汇总到 `'experiment_summary.json'`，调参实验汇总到 `'hyperparameter_tuning_summary.json'`。因此，本实验的完整流程实际上是“数据读取与张量化 - 模型训练 - 每轮验证评估 - 按验证集 F1 选最优参数 - 回滚最佳参数 - 测试集汇报结果 - 统一保存结果文件”，后续图表和报告内容也都是直接从这些结构化结果文件中自动生成的。

3 实验结果展示

3.1 训练过程与收敛特征

为展示不同结构在训练阶段的指标变化特征，下面给出了各模型验证集 Accuracy 和 F1 随训练轮次的变化过程。由于课程评价重点是 Accuracy 与 F1，因此这里直接围绕验证指标作图，更便于判断模型何时达到最佳泛化效果。

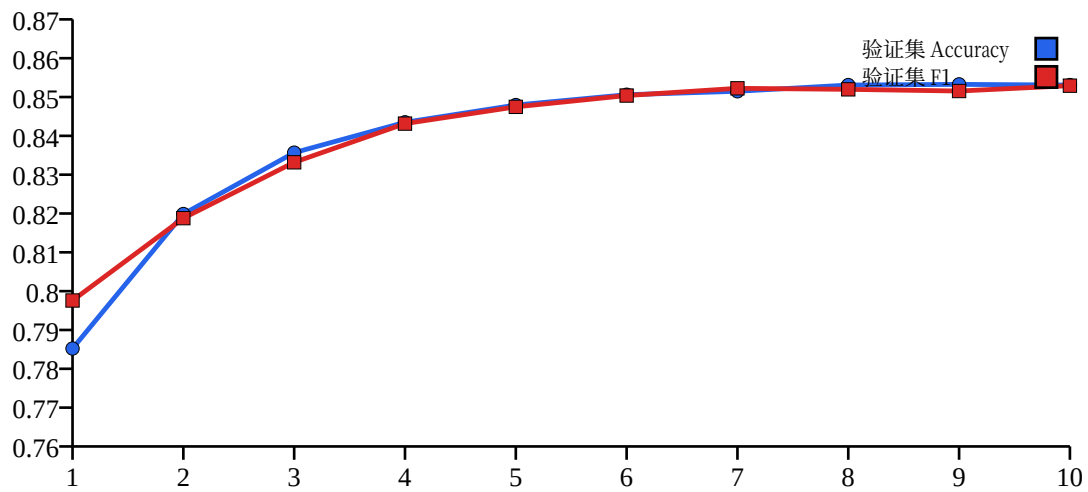


图5 MLP验证集 Accuracy 与 F1 变化曲线

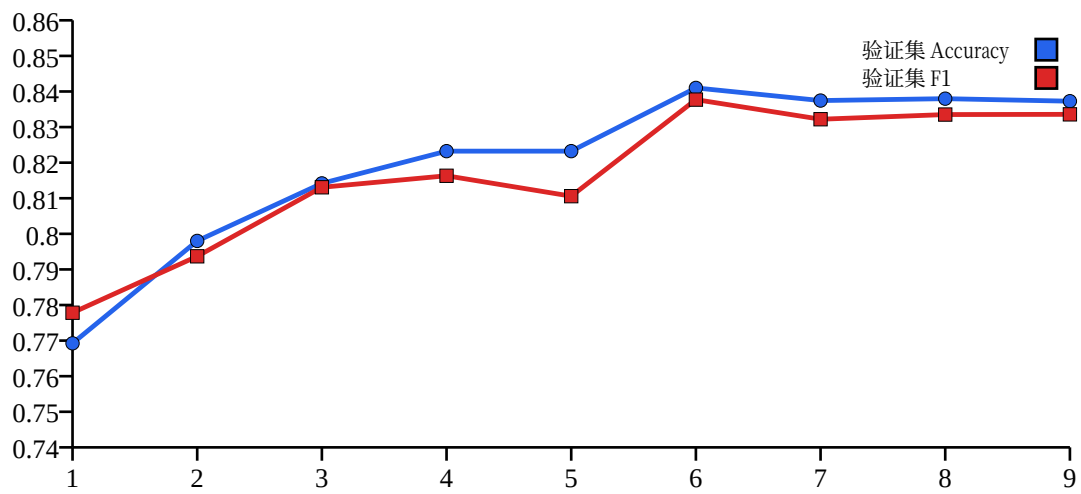


图6 TextCNN验证集 Accuracy 与 F1 变化曲线

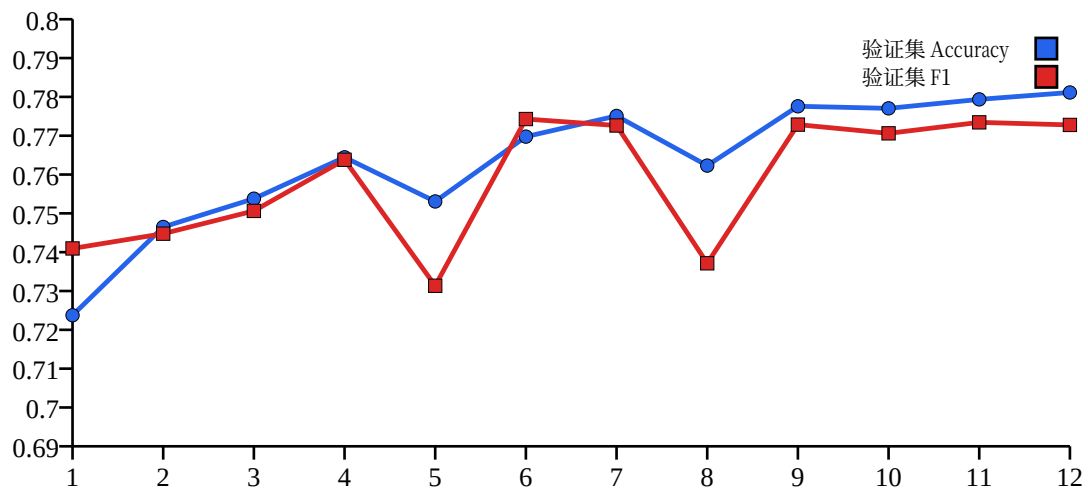


图7 BiRNN验证集 Accuracy 与 F1 变化曲线

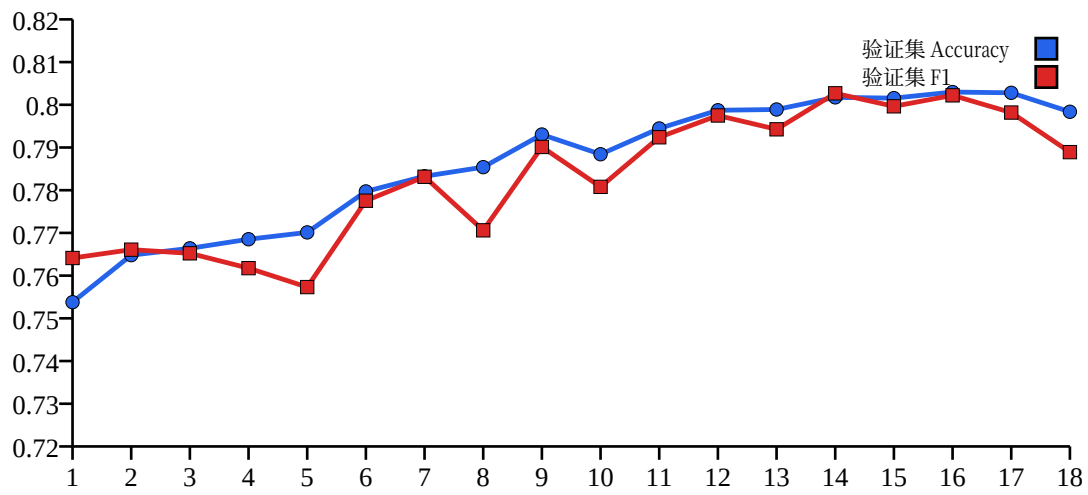


图8 BiLSTM验证集 Accuracy 与 F1 变化曲线

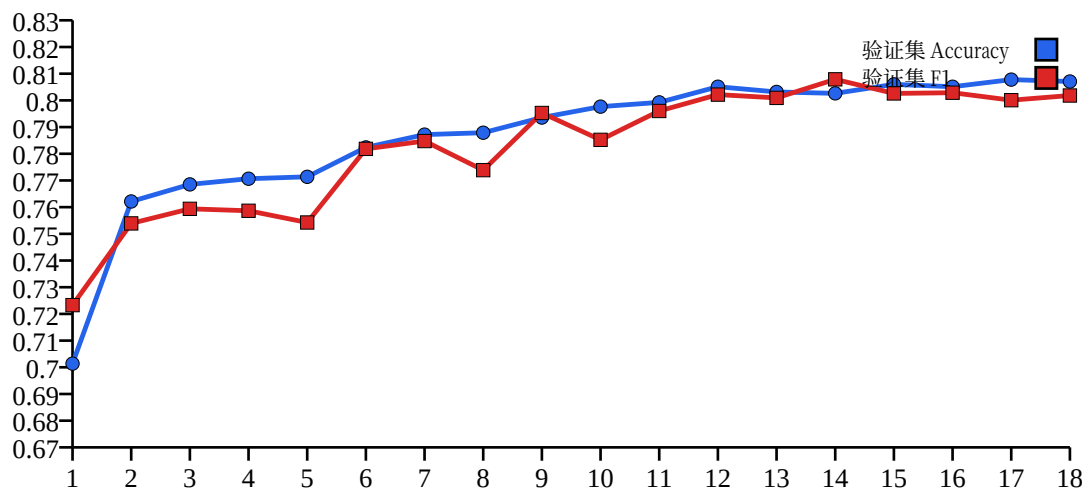


图9 BiGRU验证集 Accuracy 与 F1 变化曲线

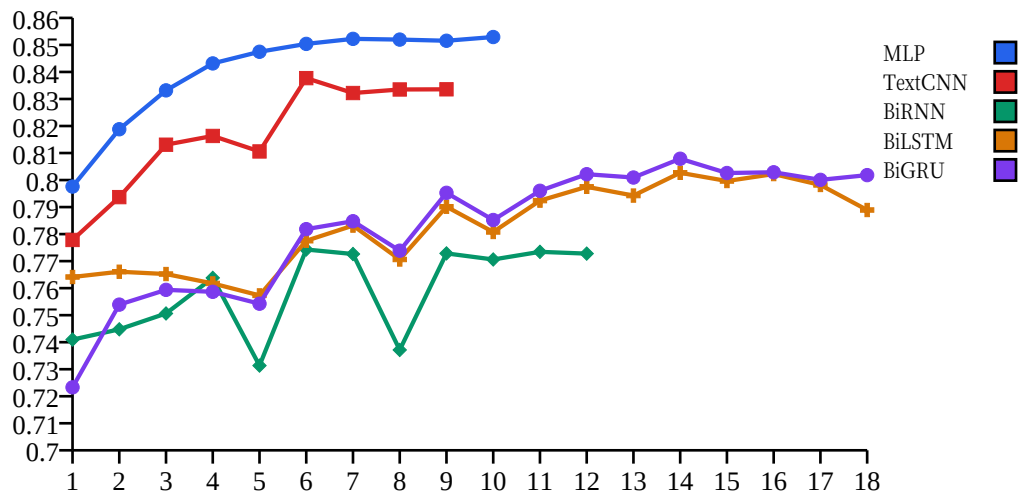


图10 各模型验证集 F1 收敛曲线

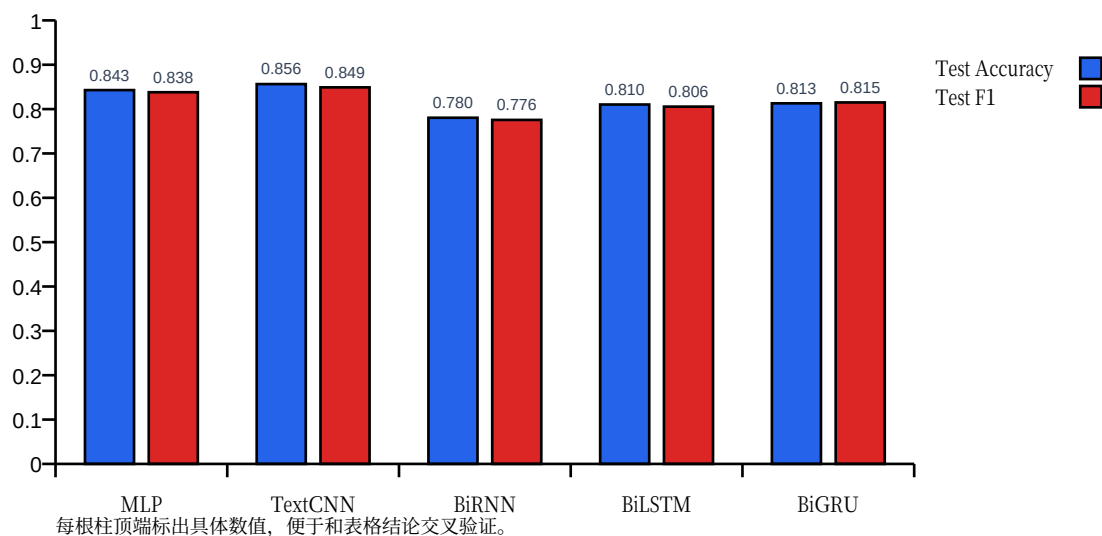


图11 主模型测试集 Accuracy 与 F1 对比

3.2 主实验结果

表3 主模型在验证集与测试集上的实验结果

模型	最佳轮次	验证集 Acc	验证集 F1	测试集 Acc	测试集 F1	参数量
MLP	10	0.8531	0.8529	0.8428	0.8380	2,673,736
TextCNN	6	0.8410	0.8377	0.8564	0.8490	2,744,904
BiRNN	6	0.7698	0.7743	0.7805	0.7756	2,713,544
BiLSTM	14	0.8017	0.8027	0.8103	0.8056	2,851,784
BiGRU	14	0.8026	0.8079	0.8130	0.8150	2,805,704

课程要求覆盖的 CNN/RNN/MLP 主实验模型都达到说明文档提出的 80% 以上指标要求，其中测试集表现最好的模型是 TextCNN，其测试集 Accuracy 为 0.8564，F1 为 0.8490。

从结果对比看，TextCNN 的测试集 F1 最高，MLP 与其非常接近；而 BiRNN 在五个主模型中相对最低。这说明当前任务的有效信息一部分来自局部短语模式，一部分来自预训练词向量本身携带的词汇极性，因此 TextCNN 和 MLP 都能取得较强结果；与此同时，带门控的 BiLSTM、BiGRU 相比普通 BiRNN 仍表现更稳，说明门控机制对长距离上下文的保留确实是有价值的。

3.3 Transformer 追加实验

当前项目中的 Transformer 采用的是“50 维静态 Word2Vec + 轻量 Transformer Encoder”方案，而不是直接微调大规模预训练语言模型。因此这里把它相关的训练预算和结果单独展开，便于和主实验中的 CNN/RNN 结论区分阅读。

表3-补充 Transformer 追加实验结果

实验	最佳轮次	词向量	验证集 Acc	验证集 F1	测试集 Acc	测试集 F1	参数量	用时/s
Transformer 正式补充实验	11	已解冻	0.7829	0.7985	0.7805	0.7896	2,811,848	349.8
Transformer 快速检查 (1轮)	1	冻结	0.7373	0.7126	0.7209	0.6831	144,898	134.6
Transformer 探测实验 (3轮)	2	已解冻	0.7506	0.7573	0.7480	0.7480	2,811,848	409.5

从现有可复核结果看，表现最好的是 Transformer 正式补充实验，其测试集 Accuracy 为 0.7805，F1 为 0.7896。与当前主实验最优的 TextCNN 相比，测试集 F1 仍有 -0.0594 的差距。基于现有 `transformer\_efficiency\_comparison.json`，1 轮 quick check 版本相对 BiLSTM 的训练速度约快 10.83x，但相对 TextCNN 仅有 0.57x，说明轻量自注意力在顺序建模上比手写循环单元更易并行，但在当前小数据、静态词向量设定下，并没有转化为更高的分类精度。

这一结果与结构本身并不矛盾：Transformer 的优势通常依赖更充足的数据规模、更深的编码器或更强的预训练语义表示，而当前实现刻意限制在单层、4 头、128 维并以静态词向量起步，更接近“把自注意力模块接到传统词向量流水线中”的工程探测，因此更适合用来展示架构差异，而不是直接挑战已充分调优的 BiLSTM/TextCNN 强基线。

4 参数对比分析

4.1 主模型参数与扩展对比实验

表4 主模型结构参数与训练参数总表

模型	结构参数	训练参数	验证集 F1	测试集 F1	参数量
MLP	hidden_dim=128 dropout=0.3	batch=128 epochs=10 lr=1e-3 wd=1e-4 patience=3 clip=0	0.8529	0.8380	2,673,736
TextCNN	num_filters=128 filter_sizes=3/4/5 dropout=0.5	batch=128 epochs=10 lr=1e-3 wd=1e-4 patience=3 clip=0	0.8377	0.8490	2,744,904
BiRNN	hidden_dim=128 num_layers=1 dropout=0.5	batch=64 epochs=18 lr=1.5e-4 emb_lr=3e-5 wd=7e-4 patience=6 clip=0.5 warmup=4 freeze_emb=4	0.7743	0.7756	2,713,544

BiLSTM	hidden_dim=128 num_layers=1 dropout=0.5	batch=64 epochs=18 lr=1.5e-4 emb_lr=3e-5 wd=7e-4 patience=6 clip=0.5 warmup=4 freeze_emb=4	0.8027	0.8056	2,851,784
BiGRU	hidden_dim=128 num_layers=1 dropout=0.5	batch=64 epochs=18 lr=1.5e-4 emb_lr=3e-5 wd=7e-4 patience=6 clip=0.5 warmup=4 freeze_emb=4	0.8079	0.8150	2,805,704
Transformer	max_len=40 model_dim=128 heads=4 layers=1 ff_dim=256 pooling=cls dropout=0.1	batch=256 epochs=12 lr=8e-4 emb_lr=8e-5 wd=1e-4 patience=4 clip=1 warmup=2	0.7985	0.7896	2,811,848

表5 扩展参数对比实验结果

对比实验	验证集 Acc	验证集 F1	测试集 Acc	测试集 F1
TextCNN (filters=64)	0.8353	0.8363	0.8699	0.8681
BiRNN (hidden=64)	0.7735	0.7798	0.7832	0.7814
BiLSTM (hidden=64)	0.8101	0.8052	0.8238	0.8159
BiGRU (hidden=64)	0.7897	0.7931	0.7913	0.7890

从附加对比实验可以看出，TextCNN 的卷积核数量从 64 提升到 128 后，局部模式提取能力更强；BiRNN、BiLSTM 和 BiGRU 的隐藏维度从 64 提升到 128 后，也更有利于保留上下文信息，但参数量和训练时间会同步增加。

为了满足“逐个超参数调节”的要求，实验进一步采用单变量调参方式：每次只修改一个超参数，其余设置保持对应基线模型不变。这里不再只展示少量示例，而是对各主模型分别列出基线参数、调参覆盖范围，并对每个超参数给出候选取值与对应指标图表。

表6 各模型基线参数与调参覆盖范围

模型	基线结构参数	基线训练参数	调参覆盖的超参数
----	--------	--------	----------

MLP	dropout=0.3 hidden_dim=128	batch_size=128 embedding_learning_rate=None epochs=10 freeze_embedding_epochs=0 grad_clip=0.0 label_smoothing=0.0 learning_rate=0.001 min_epochs=1 min_learning_rate_scale=0.2 patience=3 scheduler=plateau seed=42 warmup_epochs=0 weight_decay=0.0001	batch_size dropout epochs hidden_dim learning_rate patience seed weight_decay
TextCNN	dropout=0.5 filter_sizes=(3, 4, 5) num_filters=128	batch_size=128 embedding_learning_rate=None epochs=10 freeze_embedding_epochs=0 grad_clip=0.0 label_smoothing=0.0 learning_rate=0.001 min_epochs=1 min_learning_rate_scale=0.2 patience=3 scheduler=plateau seed=42 warmup_epochs=0 weight_decay=0.0001	batch_size dropout epochs filter_sizes learning_rate num_filters patience seed weight_decay
BiRNN	dropout=0.5 hidden_dim=128 num_layers=1	batch_size=64 embedding_learning_rate=3e-05 epochs=18 freeze_embedding_epochs=4 grad_clip=0.5 label_smoothing=0.1 learning_rate=0.00015 min_epochs=7 min_learning_rate_scale=0.15 patience=6 scheduler=warmup_cosine seed=42 warmup_epochs=4 weight_decay=0.0007	hidden_dim num_layers
BiLSTM	dropout=0.5 hidden_dim=128 num_layers=1	batch_size=64 embedding_learning_rate=3e-05 epochs=18 freeze_embedding_epochs=4 grad_clip=0.5 label_smoothing=0.1 learning_rate=0.00015 min_epochs=7 min_learning_rate_scale=0.15 patience=6 scheduler=warmup_cosine seed=42 warmup_epochs=4 weight_decay=0.0007	hidden_dim num_layers
BiGRU	dropout=0.5 hidden_dim=128 num_layers=1	batch_size=64 embedding_learning_rate=3e-05 epochs=18 freeze_embedding_epochs=4 grad_clip=0.5 label_smoothing=0.1 learning_rate=0.00015 min_epochs=7 min_learning_rate_scale=0.15 patience=6 scheduler=warmup_cosine seed=42 warmup_epochs=4 weight_decay=0.0007	hidden_dim

表7 单变量超参数调节结果总表

超参数	代表模型	调整值	测试集 Acc	测试集 F1	相对基线 $\Delta F1$
hidden_dim	MLP	64	0.8537	0.8475	-0.0144
hidden_dim	MLP	96	0.8537	0.8457	-0.0162
hidden_dim	MLP	192	0.8537	0.8492	-0.0127
hidden_dim	MLP	256	0.8537	0.8492	-0.0127
dropout	MLP	0.1	0.8537	0.8483	-0.0136
dropout	MLP	0.2	0.8591	0.8556	-0.0063
dropout	MLP	0.4	0.8564	0.8515	-0.0103
dropout	MLP	0.5	0.8591	0.8547	-0.0071
batch_size	MLP	32	0.8482	0.8486	-0.0132
batch_size	MLP	64	0.8455	0.8438	-0.0180
batch_size	MLP	192	0.8455	0.8412	-0.0207
batch_size	MLP	256	0.8455	0.8376	-0.0243
learning_rate	MLP	0.0003	0.8509	0.8433	-0.0186
learning_rate	MLP	0.0005	0.8509	0.8442	-0.0177
learning_rate	MLP	0.0015	0.8482	0.8436	-0.0183
learning_rate	MLP	0.002	0.8455	0.8412	-0.0207
weight_decay	MLP	0.0	0.8509	0.8459	-0.0159
weight_decay	MLP	5e-05	0.8509	0.8459	-0.0159
weight_decay	MLP	0.0003	0.8509	0.8459	-0.0159
weight_decay	MLP	0.0005	0.8482	0.8436	-0.0183
epochs	MLP	6	0.8455	0.8403	-0.0215
epochs	MLP	8	0.8455	0.8403	-0.0215
epochs	MLP	12	0.8455	0.8403	-0.0215
epochs	MLP	14	0.8428	0.8380	-0.0239
patience	MLP	1	0.8428	0.8380	-0.0239
patience	MLP	2	0.8374	0.8324	-0.0295
patience	MLP	4	0.8428	0.8380	-0.0239
patience	MLP	5	0.8428	0.8389	-0.0230
seed	MLP	7	0.8509	0.8493	-0.0126
seed	MLP	21	0.8482	0.8462	-0.0157
seed	MLP	84	0.8455	0.8430	-0.0189
seed	MLP	123	0.8428	0.8389	-0.0230
num_filters	TextCNN	64	0.8455	0.8430	-0.0060
num_filters	TextCNN	96	0.8347	0.8291	-0.0199

num_filters	TextCNN	160	0.8482	0.8462	-0.0028
num_filters	TextCNN	192	0.8482	0.8453	-0.0037
filter_sizes	TextCNN	(2, 3, 4)	0.8618	0.8603	+0.0113
filter_sizes	TextCNN	(2, 3, 5)	0.8564	0.8524	+0.0034
filter_sizes	TextCNN	(3, 5, 7)	0.8537	0.8525	+0.0035
filter_sizes	TextCNN	(2, 4, 6)	0.8672	0.8658	+0.0168
dropout	TextCNN	0.2	0.8591	0.8587	+0.0097
dropout	TextCNN	0.3	0.8509	0.8533	+0.0043
dropout	TextCNN	0.4	0.8482	0.8478	-0.0012
dropout	TextCNN	0.7	0.8401	0.8392	-0.0098
batch_size	TextCNN	32	0.8184	0.8241	-0.0249
batch_size	TextCNN	64	0.8537	0.8533	+0.0043
batch_size	TextCNN	192	0.8347	0.8329	-0.0161
batch_size	TextCNN	256	0.8428	0.8398	-0.0092
learning_rate	TextCNN	0.0003	0.8266	0.8232	-0.0258
learning_rate	TextCNN	0.0005	0.8537	0.8500	+0.0010
learning_rate	TextCNN	0.0015	0.8293	0.8283	-0.0207
learning_rate	TextCNN	0.002	0.8266	0.8298	-0.0192
weight_decay	TextCNN	0.0	0.8537	0.8525	+0.0035
weight_decay	TextCNN	5e-05	0.8293	0.8283	-0.0207
weight_decay	TextCNN	0.0003	0.8266	0.8261	-0.0229
weight_decay	TextCNN	0.0005	0.8320	0.8324	-0.0166
epochs	TextCNN	6	0.8455	0.8403	-0.0087
epochs	TextCNN	8	0.8266	0.8280	-0.0210
epochs	TextCNN	12	0.8347	0.8382	-0.0108
epochs	TextCNN	14	0.8374	0.8413	-0.0077
patience	TextCNN	1	0.8509	0.8451	-0.0039
patience	TextCNN	2	0.8428	0.8474	-0.0016
patience	TextCNN	4	0.8320	0.8287	-0.0203
patience	TextCNN	5	0.8266	0.8298	-0.0192
seed	TextCNN	7	0.8455	0.8412	-0.0078
seed	TextCNN	21	0.8184	0.8091	-0.0399
seed	TextCNN	84	0.8157	0.8111	-0.0379
seed	TextCNN	123	0.8293	0.8245	-0.0245
hidden_dim	BiRNN	64	0.7588	0.7548	-0.0208

hidden_dim	BiRNN	96	0.7859	0.7762	+0.0006
hidden_dim	BiRNN	160	0.7886	0.7947	+0.0191
hidden_dim	BiRNN	192	0.7913	0.7781	+0.0025
num_layers	BiRNN	2	0.8049	0.8000	+0.0244
num_layers	BiRNN	3	0.8184	0.8278	+0.0521
hidden_dim	BiLSTM	64	0.8157	0.8090	+0.0034
hidden_dim	BiLSTM	96	0.8076	0.8033	-0.0022
hidden_dim	BiLSTM	160	0.8157	0.8101	+0.0045
hidden_dim	BiLSTM	192	0.8076	0.7977	-0.0078
num_layers	BiLSTM	2	0.8374	0.8352	+0.0296
num_layers	BiLSTM	3	0.8374	0.8378	+0.0323
num_layers	BiLSTM	4	0.7724	0.7857	-0.0198
hidden_dim	BiGRU	64	0.8184	0.8174	-0.0100
hidden_dim	BiGRU	192	0.8022	0.8133	-0.0141

表8 各模型族单参数最优取值汇总

模型	超参数	基线值	最优值	最优 Val F1	对应 Test F1
BiGRU	hidden_dim	128	128	0.8279	0.8274
BiLSTM	hidden_dim	128	128	0.8027	0.8056
BiLSTM	num_layers	1	3	0.8199	0.8378
BiRNN	hidden_dim	128	192	0.7962	0.7781
BiRNN	num_layers	1	3	0.8150	0.8278
TextCNN	batch_size	128	64	0.8476	0.8533
TextCNN	dropout	0.5	0.2	0.8465	0.8587
TextCNN	epochs	10	6	0.8444	0.8403
TextCNN	filter_sizes	(3, 4, 5)	(2, 3, 4)	0.8536	0.8603
TextCNN	learning_rate	1e-03	5e-04	0.8388	0.8500
TextCNN	num_filters	128	64	0.8538	0.8430
TextCNN	patience	3	1	0.8413	0.8451
TextCNN	seed	42	42	0.8377	0.8490
TextCNN	weight_decay	1e-04	0	0.8439	0.8525
MLP	batch_size	128	256	0.8575	0.8376
MLP	dropout	0.3	0.1	0.8570	0.8483
MLP	epochs	10	6	0.8532	0.8403
MLP	hidden_dim	128	256	0.8569	0.8492

MLP	learning_rate	1e-03	5e-04	0.8578	0.8442
MLP	patience	3	5	0.8515	0.8389
MLP	seed	42	84	0.8532	0.8430
MLP	weight_decay	1e-04	5e-05	0.8541	0.8459

从单变量调参结果看，最有效的改动是把 BIRNN 的 num_layers 调到 3，使验证集 F1 相应对应基线提升了 +0.0407；而退化最明显的改动使验证集 F1 下降了 -0.0314。这里把测试集指标仅作为对应参考展示，而不用于选择超参数，目的是避免把测试集信息泄漏到调参过程。

4.1.1 单参数最优值原因分析

这里需要特别说明，表8中的“最优值”统一按验证集 F1 选取，而不是按测试集指标倒推。原因是测试集只能用于最终汇报，不能反向参与调参，否则会造成信息泄漏。因此，下文分析的“为什么这个值最好”，本质上是在解释：为什么它在当前单变量实验设置下给出了更高的验证集泛化表现，而不是单纯追求某一次测试集分数的偶然峰值。

从 MLP 的结果看，最优 hidden_dim=256、batch_size=256、dropout=0.1、learning_rate=5e-04、epochs=6、patience=5、weight_decay=5e-05、seed=84。这组结果说明：对“平均池化后再分类”的 MLP 来说，当前 baseline 的正则和步长都略偏保守。hidden_dim 从 128 提到 256 后，验证集 F1 从 0.8480 升到 0.8569，说明平均句向量本身信息量并不算低，增加分类头容量后，模型能把这些语义更充分地投影到判别空间里；batch_size 选到 256，则说明在当前数据规模和优化器设置下，更平滑的梯度估计反而更有利于稳定收敛。与此同时，dropout 从 0.3 降到 0.1、weight_decay 从 1e-04 降到 5e-05 反而更优，说明这一路模型并不是过强容量导致过拟合，而更像是 baseline 正则稍强，压制了分类层对有效情感特征的利用。learning_rate 选到 5e-04 而不是 1e-03，也支持同样判断：更小的步长虽然收敛慢一点，但能让浅层分类器在预训练词向量上更稳定地调整决策边界。epochs=6 和 patience=5 则表明 MLP 不需要特别长的训练预算，继续训练未必能换来泛化提升；seed=84 最优更多说明 MLP 对随机初始化有一定敏感性，因此后续若要给出更稳妥结论，最好再做多 seed 平均，而不能只盯某一次运行。

TextCNN 的最优组合是 filter_sizes=(2, 3, 4)、num_filters=64、batch_size=64、dropout=0.2、learning_rate=5e-04、epochs=6、patience=1、weight_decay=0，而 seed 仍以 42 为最好。这个结果非常有解释价值。首先，filter_sizes 从 baseline 的 (3, 4, 5) 调成 (2, 3, 4) 后，验证集 F1 提升最明显之一，说明这份影评数据里的关键情感证据更集中在较短

局部模式上，例如“很失望”“不推荐”“超出预期”这类2到4个词的组合比更长的5-gram更常承担判别作用。其次，num_filters从128降到64反而更好，说明baseline的卷积通道数略显冗余；在当前数据规模下，过多filter并没有持续带来新模式，反而可能把一些噪声局部模式也学进去。batch_size=64和learning_rate=5e-04共同最优，则表明CNN需要比baseline更稳的梯度更新：batch太小会让训练波动过大，太大又会削弱泛化，而64在这次实验里恰好落在较合适的中间位置。dropout从0.5降到0.2、weight_decay从1e-04降到0、epochs=6且patience=1的组合，则进一步说明CNN在这份任务上收敛很快，baseline更像是“正则偏强、训练略长”，因此适当放松正则、缩短训练、让模型更早在验证集最好处停下来，反而更容易保住局部模式提取带来的优势。seed=42仍为最优，也意味着CNN在不同随机种子下虽有波动，但baseline的初始化并不差。

BiRNN的最优hidden_dim=192、num_layers=3。其中提升最明显的是num_layers：从baseline的1层增加到3层后，验证集F1从0.7743提升到0.8150，说明对不带门控的普通RNN来说，单层表示能力偏弱，增加堆叠深度比单纯扩大状态维度更能补足建模能力。hidden_dim从128提高到192也有提升，但幅度明显小于层数，而且测试集并没有同步放大收益，这意味着“加宽”虽然增强了容量，却也带来了更高的不稳定性。因此，这一轮单变量调参给出的核心结论不是“BiRNN越大越好”，而是“普通循环单元先天表达力有限时，适度增加层数比盲目增大隐藏维更有效”。这也从侧面解释了为什么后续BiLSTM往往更容易取得稳定优势：与其不断给vanilla RNN叠容量，不如直接换成门控更强的单元。

BiGRU这组结果反而很有代表性：hidden_dim的最优值就是baseline的128，两边无论降到64还是升到192，验证集F1都下降。这通常说明当前GRU的容量已经处在比较合适的位置。减小到64时，模型对上下文的压缩能力不够，容易漏掉情感细节，属于欠拟合；增大到192时，参数量和优化难度一起上升，但在当前训练预算和数据规模下并没有换来更好的泛化，反而出现收益回落，更接近“容量增加但没有被有效利用”。换句话说，门控机制已经让GRU在128维时具备较好的表达效率，因此这一轮调参更像是在告诉我们：对于GRU，这个任务上“保持适中容量”比继续堆宽度更重要。

综合来看，这些“最优值”并不是彼此独立的随机结果，而是共同反映了当前任务的数据特点和训练条件：影评情感线索以短局部模式为主，因此CNN更偏好较短卷积窗口；平均池化后的MLP容量不必太小，但正则不宜过重；普通

BiRNN 若不引入门控，就更依赖增加层数来补足表示能力；而 GRU 由于门控效率更高，反而不需要继续盲目扩宽。也要强调，单变量调参得到的是局部最优判断，而不是全局联合最优解，因此这些结论最适合指导下一轮更有针对性的联合调参，而不应被理解为放之四海皆准的固定规则。

4.2 逐个超参数图表与参数含义说明

为便于解读后续单变量调参图表，下面先说明本实验实际调到的超参数分别控制什么、其背后的训练原理是什么，以及取值偏小或偏大时通常会带来哪些现象。这样在阅读曲线时，就能把“指标变化”与“参数作用机制”对应起来，而不是只停留在经验比较层面。

`hidden_dim`：影响隐藏表示的容量。在 MLP 中它是中间全连接层的宽度，在 BiRNN/BiGRU 中它是每个方向的隐藏状态维度；本质上决定模型能用多大的特征空间去压缩句子信息。取值偏小时，模型表达能力不足，容易漏掉细粒度情感线索，表现为欠拟合；取值偏大时，参数量、显存和训练时间都会增加，也更容易把训练集噪声记进去，导致过拟合。

`num_layers`：影响循环编码器的深度。增加层数相当于让高层在低层时序表示之上再做一次抽象，理论上能学习更复杂的上下文组合。层数过少时，模型只能进行较浅的序列变换；层数过多时，优化难度明显上升，训练更慢，也更容易出现梯度传播不稳定和收益递减。

`num_filters`：影响 TextCNN 每种卷积核尺寸下可学习的局部模式探测器数量。每个 filter 都可以理解为一种“情感短语模板”；数量偏少时，可覆盖的局部模式有限，容易漏掉不同表达方式；数量偏多时，虽然模式覆盖更广，但参数量会同步增加，带来更高的过拟合风险和更慢的训练。

`filter_sizes`：影响卷积核沿时间轴一次观察多少个相邻 token，也就是 n-gram 感受野大小。较小的窗口更擅长抓“好看”“失望”这类短局部模式；较大的窗口更容易覆盖“并不推荐”“一点也不好”这类更长搭配。若窗口整体偏小，模型可能看不到较长的否定或转折结构；若窗口整体偏大，则容易把过多无关上下文卷进去，对短句尤其可能造成特征稀释。

`dropout`：影响训练时随机屏蔽激活单元的比例，其原理是打破特征之间的过强共适应，从而提升泛化能力。`dropout` 偏小时，正则化不足，模型更容易记住训练集；`dropout` 偏大时，有效信息会被丢掉太多，优化变难，训练和验证指标都可能一起下降，表现为欠拟合。

`learning_rate`：影响 AdamW 每次参数更新的步长，是最直接决定收敛速度与稳定性的训练超参数。学习率偏小时，每一步移动太保守，有限训练轮数内可能还没走到较优区域；学习率偏大时，参数会在最优点附近来回震荡，严重时甚至直接发散，导致验证集指标不稳定。

`batch_size`：影响每次梯度更新使用多少条样本。`batch` 较小时，梯度噪声更大，更新更频繁，往往带来一定正则化效果，但训练波动也更明显；`batch` 较大时，梯度估计更平滑、吞吐更高，但更新次数减少，且有时会削弱泛化能力。过小可能让训练不稳定、耗时增加；过大则可能让模型更容易停在较“平”的次优点，或受显存与内存限制。

`weight_decay`：影响 L2 正则化强度。它通过惩罚过大的权重，抑制模型把少量训练样本拟合得过于极端。`weight decay` 偏小时，约束不足，模型更容易过拟合；偏大时，权重会被压得过小，模型难以形成足够强的判别边界，表现为欠拟合。

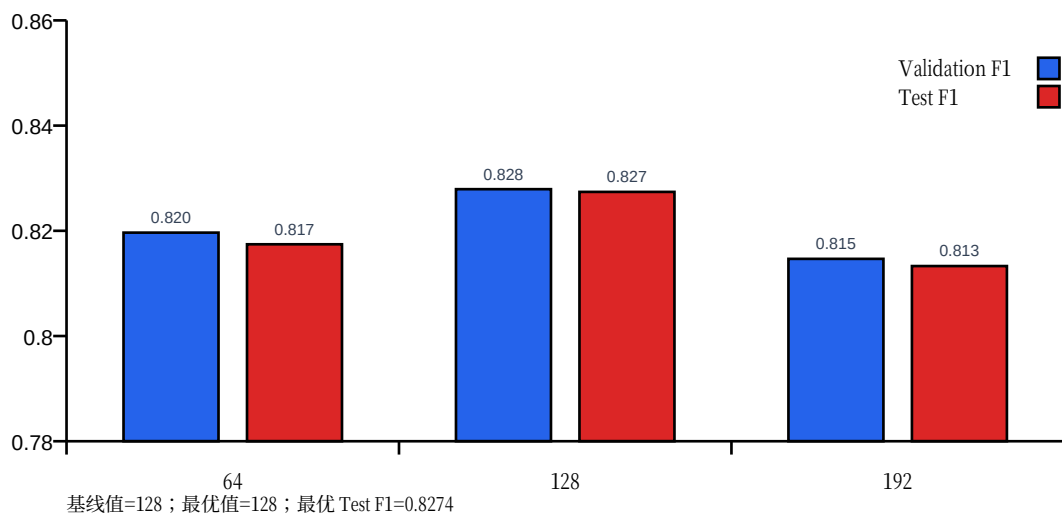
`epochs`：影响完整遍历训练集的最大轮数，也就是给模型多少优化预算。轮数偏小时，模型可能还没学到稳定的判别模式就停止训练；轮数偏大时，虽然在有早停时风险可控，但仍会增加训练时间，并在早停不敏感时放大过拟合的可能。

`patience`：影响早停策略的容忍度，即验证集指标连续多少轮不提升后才停止训练。`patience` 偏小时，模型可能在正常波动阶段就被提前终止，错过后续继续提升；`patience` 偏大时，虽然更稳妥，但也会拉长训练时间，甚至让模型在无效轮次中继续朝训练集过拟合。

`seed`：影响随机初始化、`batch` 打乱顺序以及部分随机训练行为。它不是“越大越好”或“越小越好”的量纲型超参数，而是用来观察实验稳定性的随机起点。不同 `seed` 可能带来略有差异的最优点；如果只看单个 `seed`，结果可能带有偶然性，因此更重要的是检查模型对 `seed` 是否敏感。

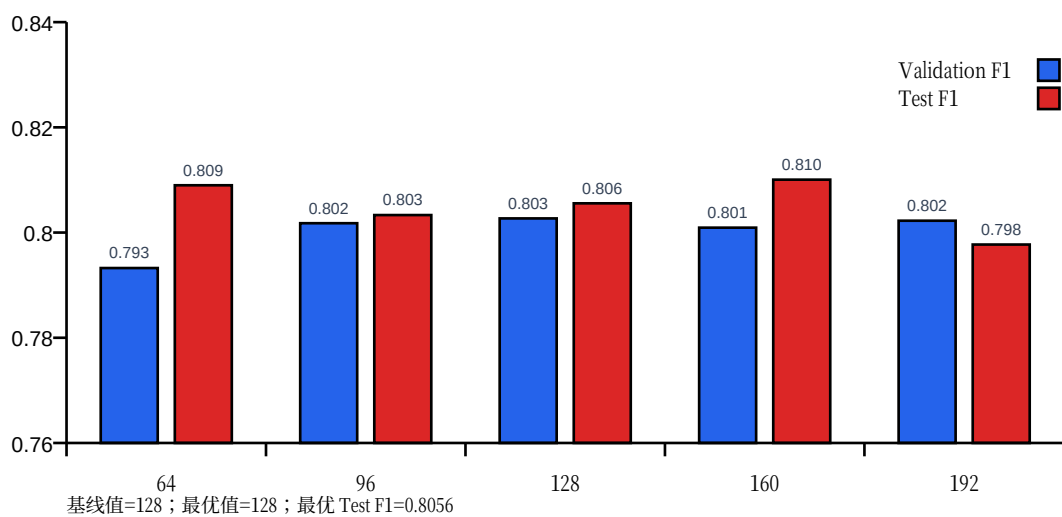
下面的图表严格采用控制变量法：每次只改变一个超参数，其余结构参数与训练参数保持对应模型基线不变。横轴为候选取值，纵轴为验证集与测试集 F1，便于直接比较不同取值下的效果差异。

BiGRU

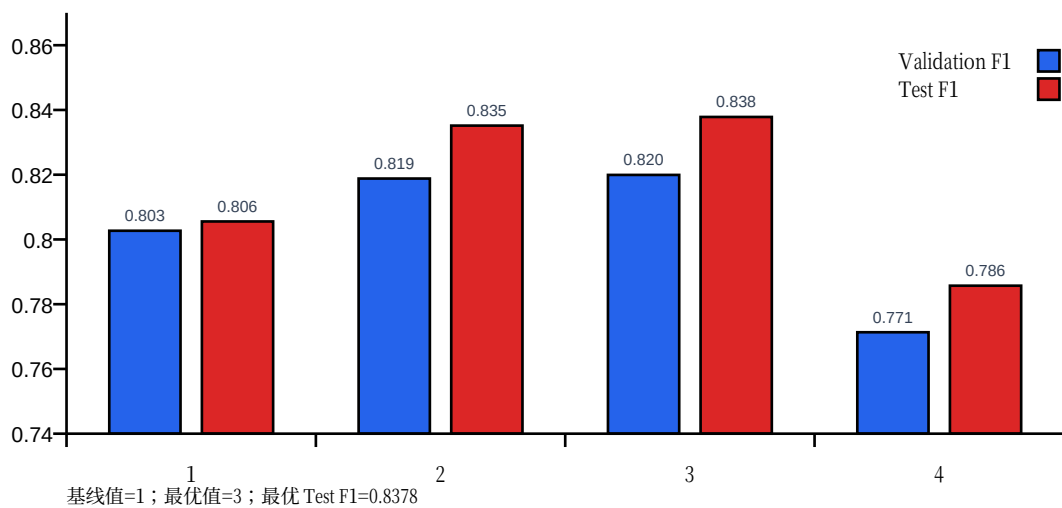


BiGRU hidden_dim 单变量调参结果

BiLSTM

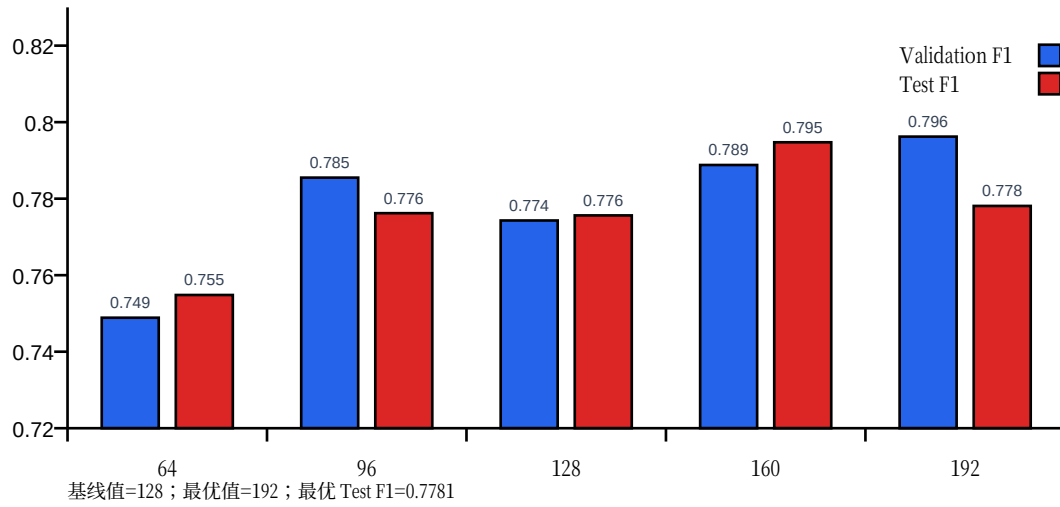


BiLSTM hidden_dim 单变量调参结果

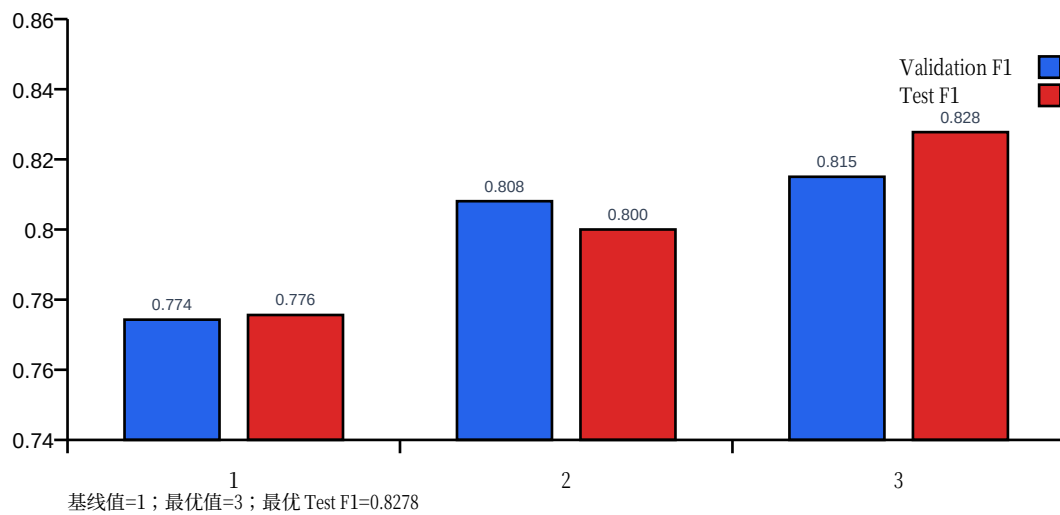


BiLSTM num_layers 单变量调参结果

BiRNN

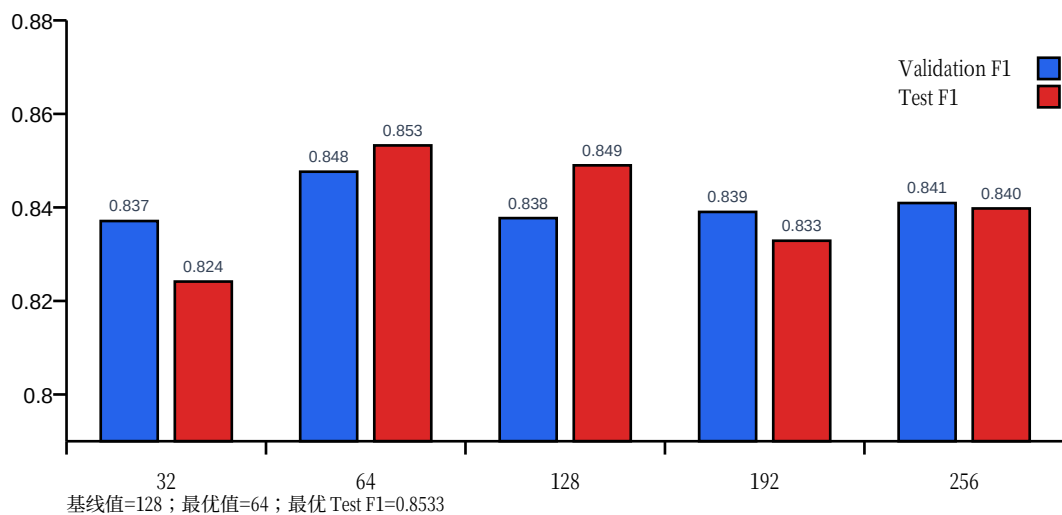


BiRNN hidden_dim 单变量调参结果

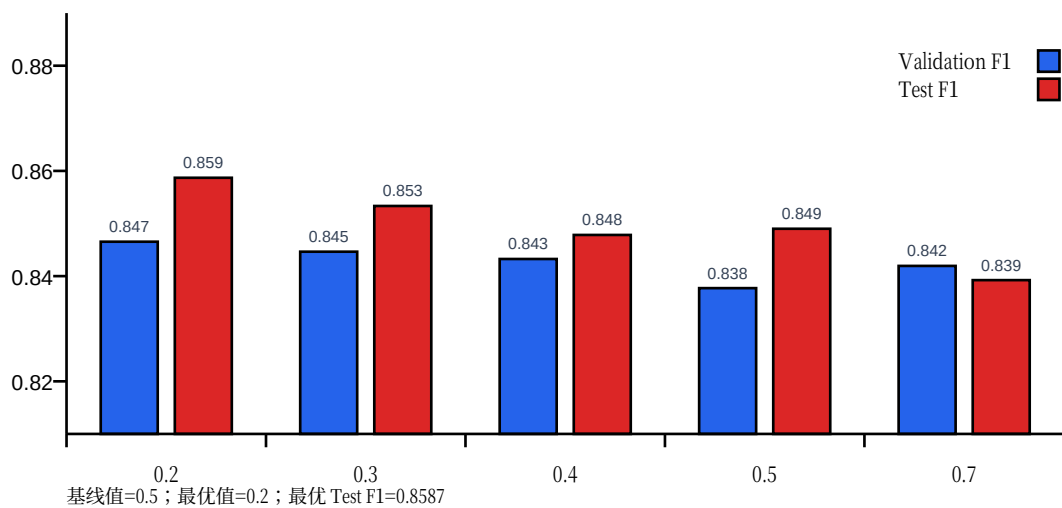


BiRNN num_layers 单变量调参结果

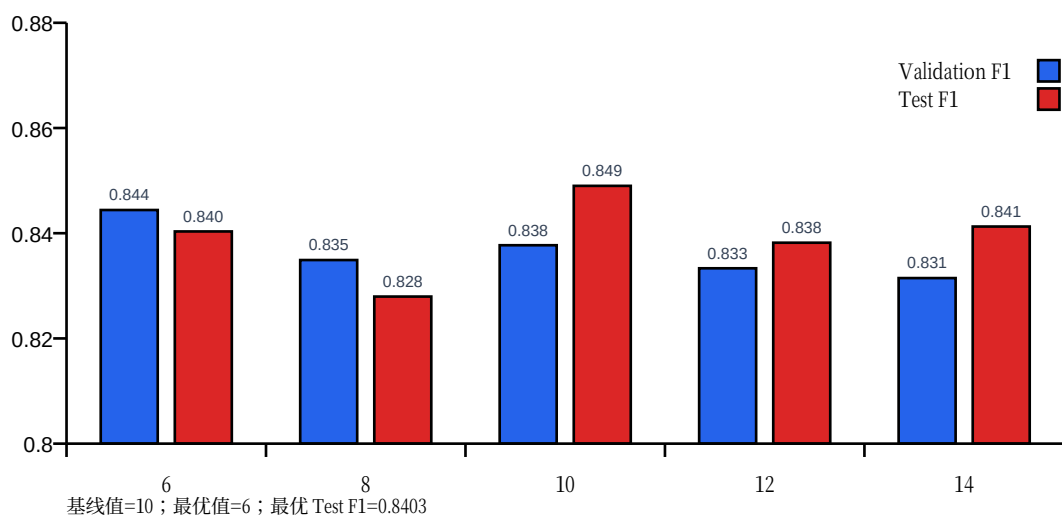
TextCNN



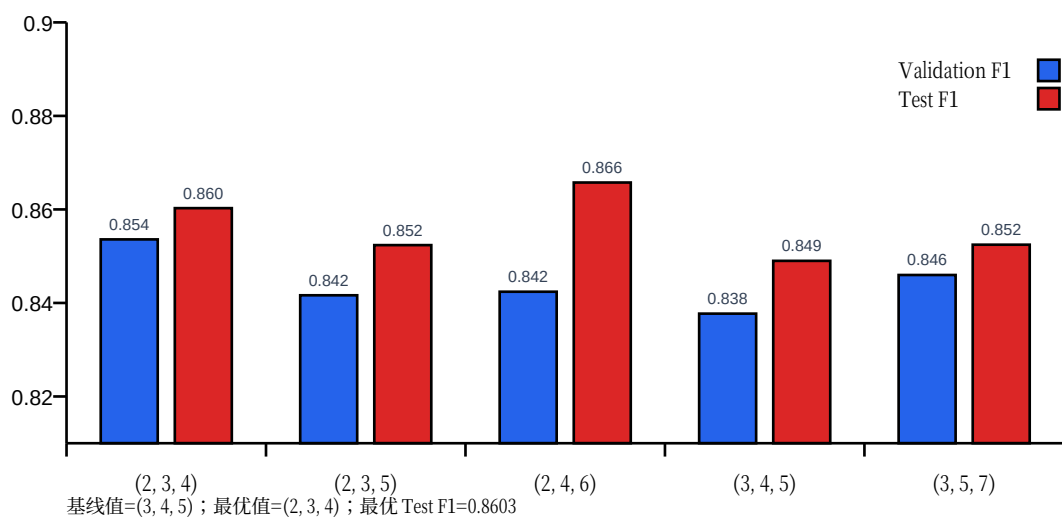
TextCNN batch_size 单变量调参结果



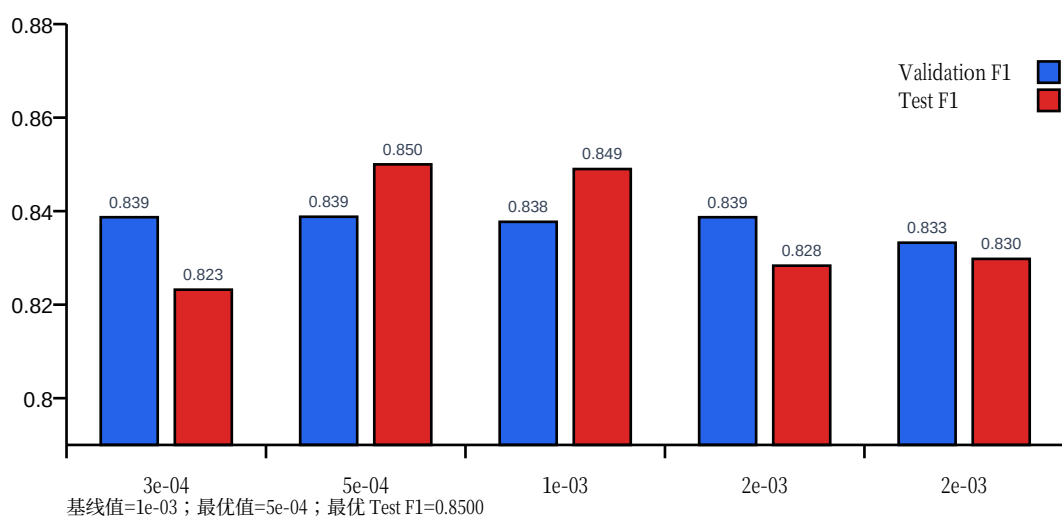
TextCNN dropout 单变量调参结果



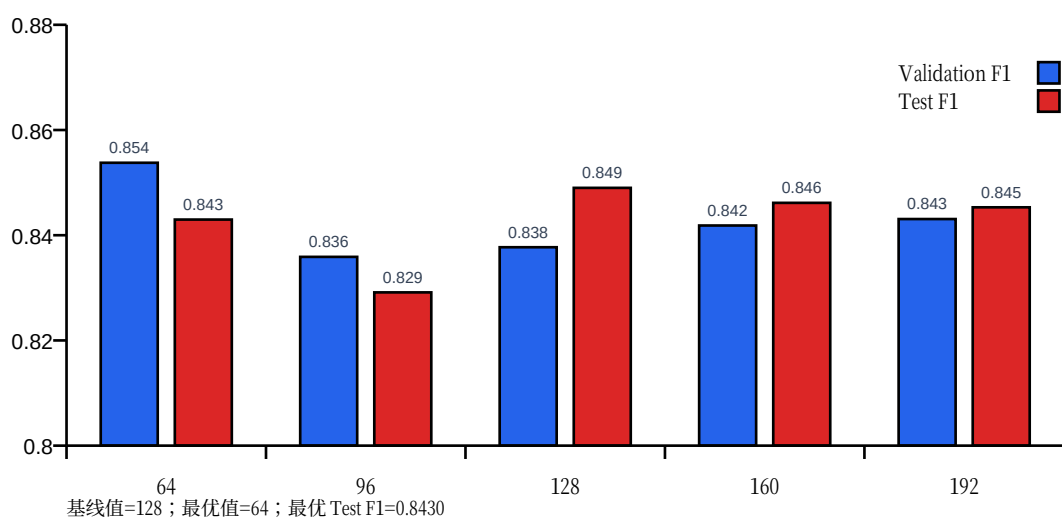
TextCNN epochs 单变量调参结果



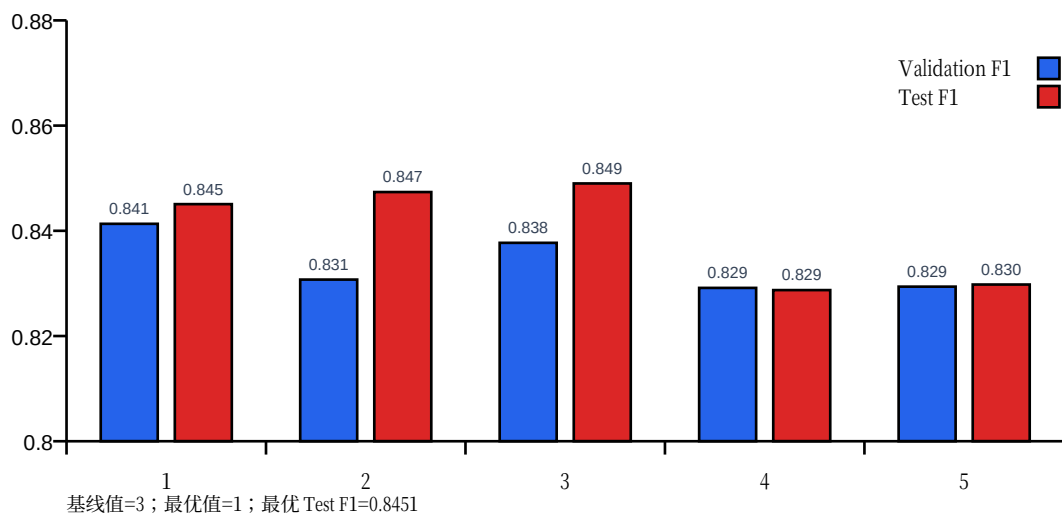
TextCNN filter_sizes 单变量调参结果



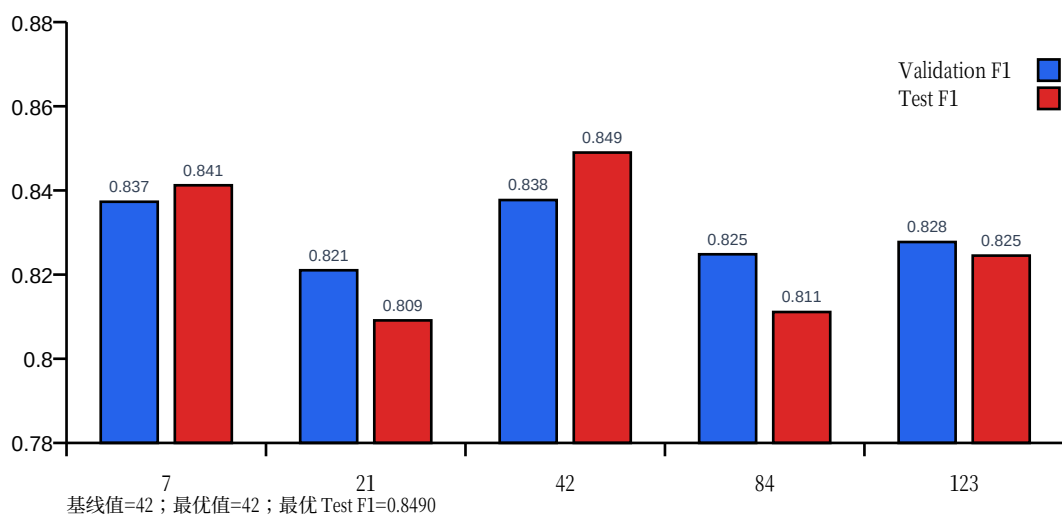
TextCNN learning_rate 单变量调参结果



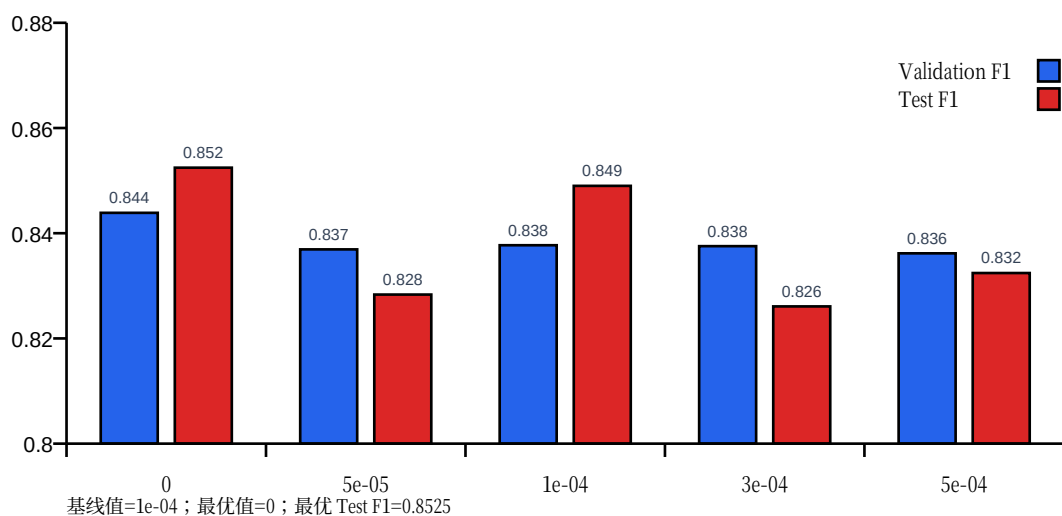
TextCNN num_filters 单变量调参结果



TextCNN patience 单变量调参结果

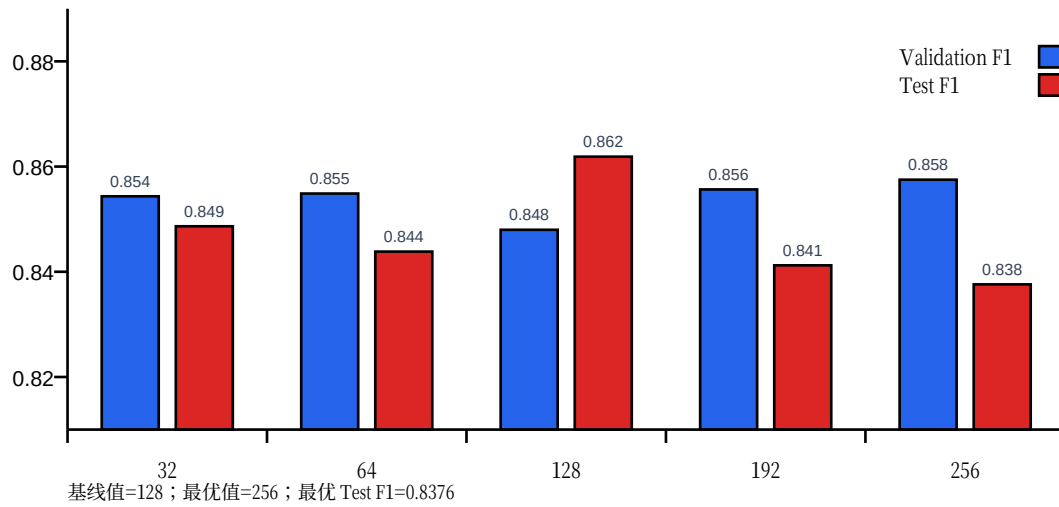


TextCNN seed 单变量调参结果

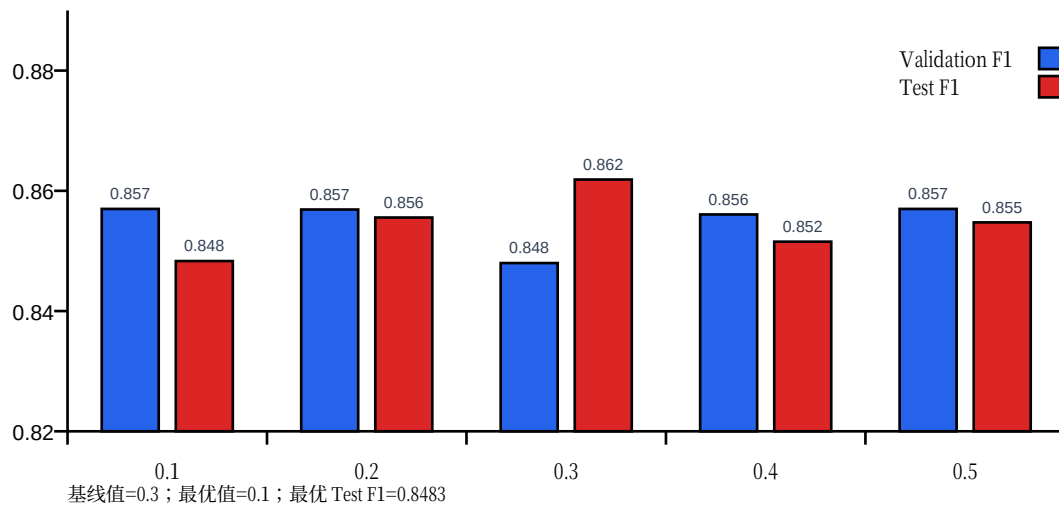


TextCNN weight_decay 单变量调参结果

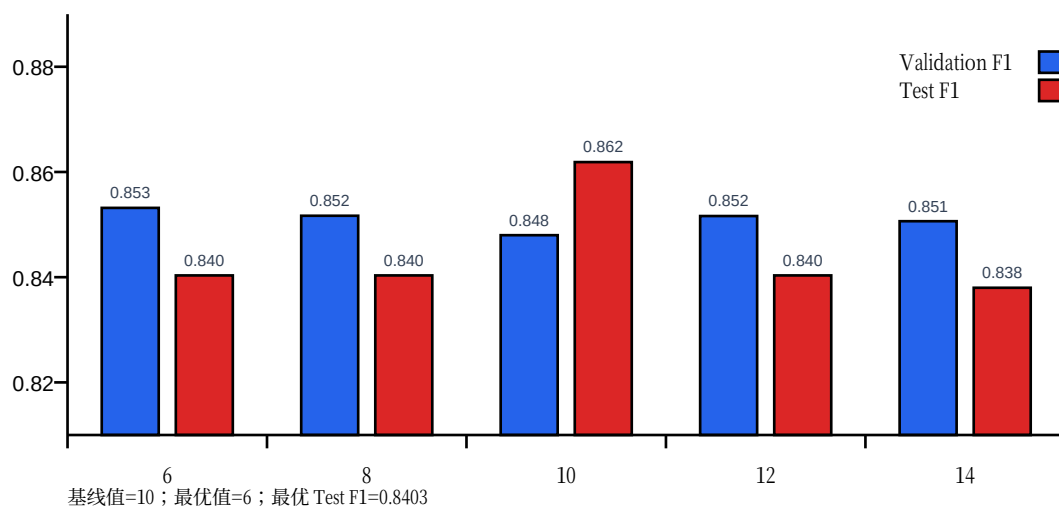
MLP



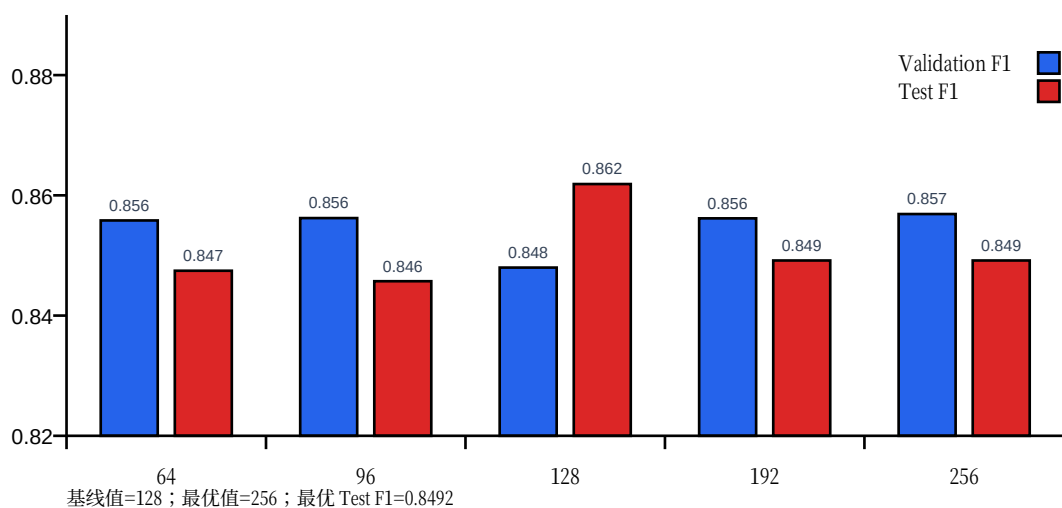
MLP batch_size 单变量调参结果



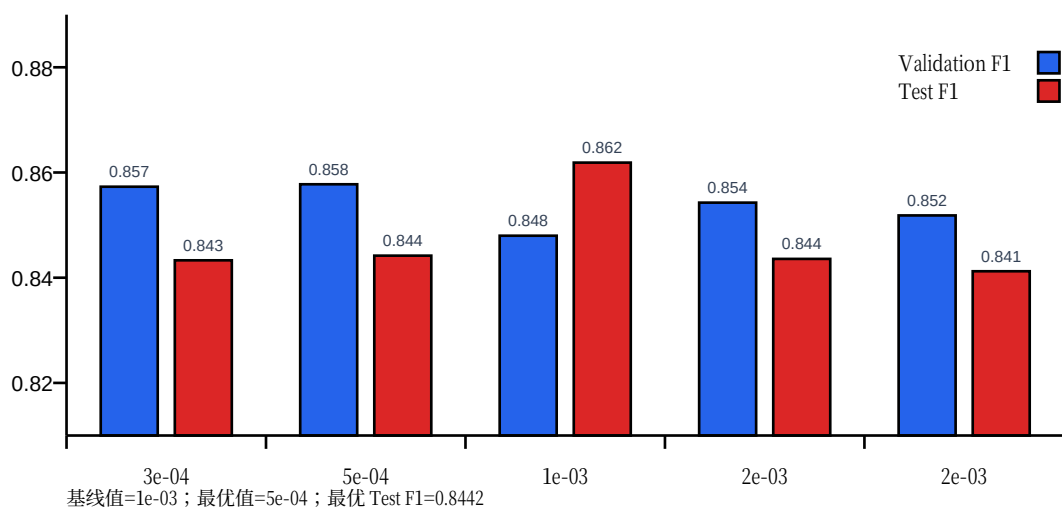
MLP dropout 单变量调参结果



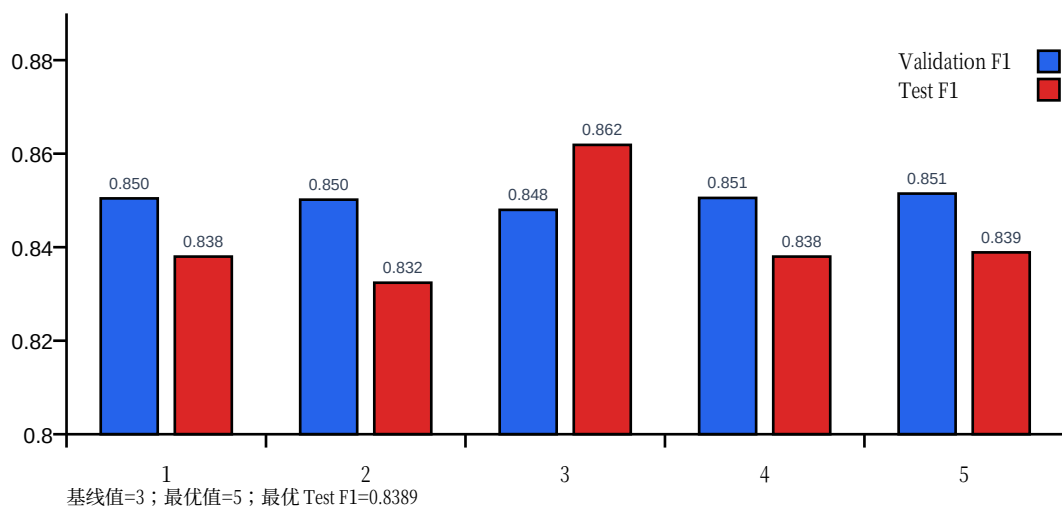
MLP epochs 单变量调参结果



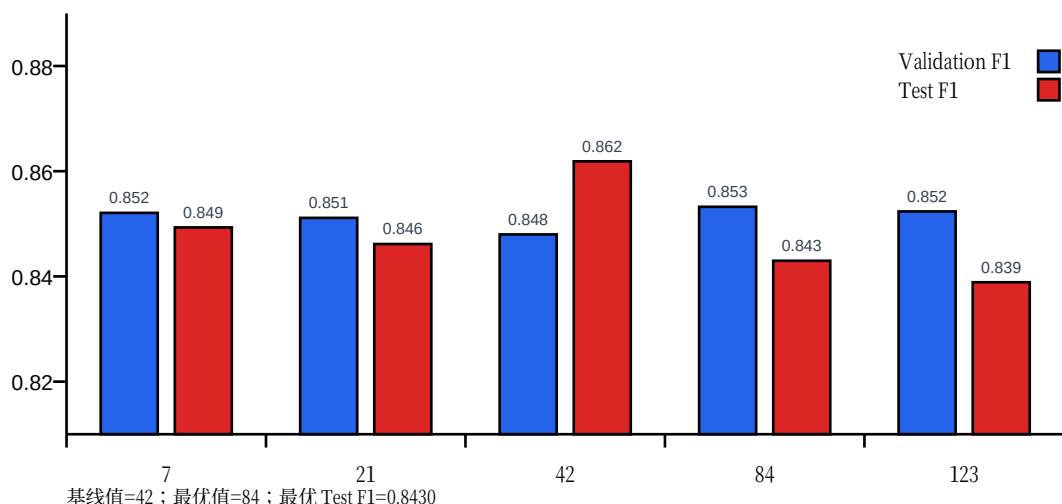
MLP hidden_dim 单变量调参结果



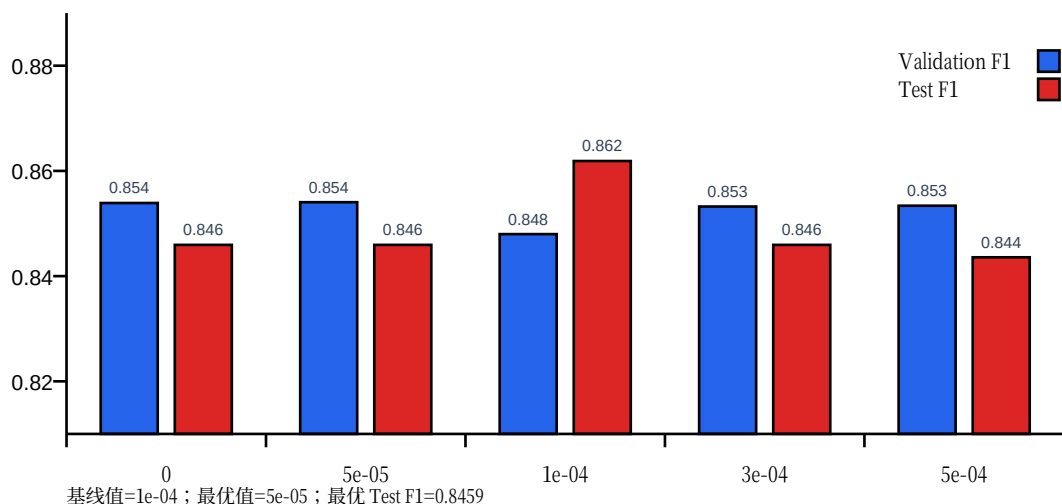
MLP learning_rate 单变量调参结果



MLP patience 单变量调参结果



MLP seed 单变量调参结果



MLP weight_decay 单变量调参结果

4.3 结构改进与数据增强追加实验

为了把“结构优化”和“额外数据增强”写成一条更清晰的实验链路，本文在主实验之外补充了两组逐步改进实验：其一是沿着‘BiRNN → BiLSTM → BiLSTM + Attention’验证循环结构升级；其二是沿着‘单卷积核 CNN → 多卷积核 TextCNN → TextCNN + 数据增强’验证卷积结构和轻量增强的收益。考虑到这些实验属于追加分析，本节结果全部来自已有指标文件的重新汇总，因此这里更适合用来判断改进方向是否值得保留，而不直接替代主实验最终排名。

表8-1 RNN 路线的逐步结构改进结果

实验	训练数据处理	验证集 F1	测试集 Acc	测试集 F1	相对上一阶段 Test F1
BiRNN 基线	无	0.7743	0.7805	0.7756	-
BiLSTM	无	0.8027	0.8103	0.8056	+0.0299

BiLSTM + Attention	无	0.7845	0.7967	0.8011	-0.0045
--------------------	---	--------	--------	--------	---------

从 RNN 路线看，`BiLSTM` 相比 `BiRNN` 的测试集 F1 提升了 +0.0299，说明在这份影评数据上，引入门控记忆机制确实有助于保留更长距离的上下文信息。相比之下，`BiLSTM + Attention` 在本轮 quick 预算下测试集 F1 相比 `BiLSTM` 下降了 -0.0045，表明注意力机制并不是“加上就涨”，仍需要继续调节 attention 维度、dropout 和训练轮数，才能更稳定地发挥作用。

表8-2 CNN 路线的结构改进与数据增强结果

实验	训练数据处理	验证集 F1	测试集 Acc	测试集 F1	相对上一阶段 Test F1
CNN 单卷积核基线	无	0.8139	0.8266	0.8140	-
TextCNN 多卷积核	无	0.8377	0.8564	0.8490	+0.0350
TextCNN + Word Dropout	Word Dropout	0.8423	0.8401	0.8392	-0.0098
TextCNN + Random Deletion	Random Deletion	0.8406	0.8320	0.8297	-0.0193
TextCNN + Synonym Replacement	Synonym Replacement	0.8408	0.8347	0.8310	-0.0180

从 CNN 路线看，`TextCNN` 相比单卷积核 CNN 的测试集 F1 提升了 +0.0350，说明多尺度卷积核更适合同时捕捉“很好看”“一点也不好看”这类不同长度的情感短语。三种轻量增强都没有超过未增强的 TextCNN，其中 `Word Dropout` 最接近基线，测试集 F1 仅变化 -0.0098；而 `Random Deletion` 与 `Synonym Replacement` 退化更明显，说明当前增强强度对这份已分词影评数据来说偏大，语义扰动已经开始覆盖其潜在收益。

5 模型比较

5.1 Baseline、CNN 与 RNN 的比较

若直接按这次主实验的测试集 F1 排序，结果为：TextCNN(F1=0.8490) > MLP(F1=0.8380) > BiGRU(F1=0.8150) > BiLSTM(F1=0.8056) > BiRNN(F1=0.7756)。其中 TextCNN 以 0.8490 的测试集 F1 排名第一，只比第二名 MLP 高 0.0110，但相对最低的 BiRNN 仍高出 0.0734。这说明当前数据集上并不存在“绝对碾压”的单一结构，更准确的判断是：不同模型各自擅长捕捉不同层次的情感线索，而最终排名取决于这些能力与数据分布的匹配程度。

MLP 通过平均池化快速聚合句向量，结构最简单，同时也是本实验里训练最快的主模型：主实验中 MLP 训练约 25.4 秒，而 TextCNN 约 166.8 秒，说明 TextCNN 的训练时间大约是 MLP 的 6.56 倍。TextCNN 对“非常失望”“剧情混乱”这类局部情感短语尤其敏感，参数共享带来较强的局部模式提取能力，但多卷积核并行和更复杂的特征拼接也会带来额外训练开销。BiRNN、BiLSTM 和 BiGRU 则显式建模上下文

顺序，更适合处理依赖前后语义的长句；但代价也很明确，它们训练时长分别约为 TextCNN 的 3.54、14.94 和 11.35 倍，其中 BiLSTM 的门控记忆最完整，BiGRU 在效果和效率之间折中得更明显。

从这次结果看，TextCNN 排名第一的直接原因，是这份中文影评数据里的有效判别信号大多集中在局部短语层面，例如“非常感人”“不值得看”“剧情混乱”“超出预期”这类 2 到 5 个词的搭配。TextCNN 的多尺度卷积核恰好适合并行扫描这些短语，再用最大池化保留最强情感证据，因此既能抓住否定结构和程度副词，又能在效果和训练成本之间维持较好的平衡。它在本次重跑中测试集 F1 达到 0.8490；虽然训练速度并不是主模型里最快的，但相较需要更长训练时间的双向循环模型，它仍然以较低成本给出了最好的测试集结果，这就是它在综合比较里最占优的原因。

MLP 能排到第二，说明这个任务并不完全依赖复杂时序建模。预训练词向量已经把不少情感词语义放到了相对合理的位置，平均池化虽然丢掉了词序，但仍能把“推荐、精彩、感人”和“拖沓、混乱、乏味”这类高频情感词的总体倾向保留下来，因此 MLP 的测试集 F1 仍有 0.8380。它没有超过 TextCNN，主要是因为平均操作会弱化“不值得推荐”“前半段很慢但是后半段精彩”这类需要局部组合或转折关系才能判断的句子；但它也没有明显输给循环模型，反而说明在当前数据规模和静态词向量设定下，很多样本靠词汇层面的极性就已经能做出较强判断。

三种双向循环模型里，BiGRU 优于 BiLSTM，BiLSTM 又明显优于 BiRNN。这和结构能力是吻合的：BiRNN 使用最基础的循环单元，长距离信息经过多步传递后更容易衰减，所以它在包含转折、让步和长句评价时更容易丢失关键上下文；BiLSTM 和 BiGRU 通过门控机制显式控制“该记什么、该忘什么”，因此测试集 F1 分别提升到 0.8056 和 0.8150。而这次 BiGRU 最终略高于 BiLSTM，一个合理解释是：当前数据规模不大、训练预算有限，GRU 用更少的门控参数换来了更稳定的优化，因而在不牺牲太多表达能力的前提下取得了更好的泛化平衡。

作为追加模型，Transformer 不再沿时间步递推隐藏状态，而是通过多头自注意力让每个 token 直接与其余有效 token 交互，因此理论上更擅长建模长距离依赖且并行度更高。不过在本项目的轻量设定下，它缺少大规模预训练语言模型带来的上下文语义先验，最终表现更像是一个“可并行的全局特征提取器”。本次正式补充实验里，它的测试集 F1 为 0.7896，低于主实验最优的 TextCNN。原因并不难理解：当前实现只有单层、4 头、128 维，而且输入仍是静态 Word2Vec；在这种设定下，自注意力虽然能建模全局关系，但缺少更强预训练语义和更深层表示能力，优势不足以转化为更高分类精度。

5.2 综合比较结论

本文基于统一的数据处理流程与评价指标，实现并比较了 MLP、TextCNN、BiRNN、BiLSTM、BiGRU 等主模型，并补充了轻量 Transformer 编码器的结构与追加实验。实验表明，预训练词向量为各类模型提供了较强的初始语义表示，而不同网络结构的差异主要体现在对局部模式、词序与上下文依赖的建模方式上。综合测试集指标、训练曲线和参数对比结果可以看到，TextCNN 在当前任务上取得了最好的测试集结果，MLP 则以最低训练成本给出了很强的竞争基线；带门控的 BiLSTM、BiGRU 相比普通 BiRNN 更稳定，但在当前数据规模下并未整体超过 TextCNN。Transformer 追加实验则说明，自注意力结构本身值得保留，但若缺少更强预训练语义或更大训练预算，其优势未必会在小规模影评任务上自然显现。

与此同时，外部微博测试也表明当前模型的跨域迁移能力较弱，说明仅依赖单一领域训练数据仍难以获得稳定的通用情感判别能力。后续若进一步引入更大规模的跨域语料、上下文增强表示或更强的预训练语言模型，模型的泛化能力仍有较大提升空间。

追加实验进一步给出了更细的工程判断：在当前任务上，优先升级核心结构比直接叠加轻量文本增强更稳妥。具体而言，‘BiRNN → BiLSTM’与‘单卷积核 CNN → 多卷积核 TextCNN’都带来了明确收益，而‘BiLSTM + Attention’和三种数据增强仍需额外调参才能超过已有强基线。这一结论也说明，新增模块的价值必须建立在合适训练预算和匹配数据分布之上。

6 问题思考回答

6.1 训练何时停止最合适

本实验采用“以验证集 F1 为核心指标的早停”策略，即每轮训练后都在验证集上计算 Accuracy 和 F1，并以 F1 是否继续提升作为是否保留当前参数的主要依据。之所以不用“训练到固定 10 轮或 20 轮就停止”的硬规则，是因为不同模型的收敛速度差异很大：有的模型很快就达到最佳泛化点，有的模型则需要更长时间才能把词向量和分类层一起调整到位。

从本次实验结果看，BiRNN、BiLSTM 和 BiGRU 的最优轮次分别出现在第 6、14、14 轮，TextCNN 出现在第 6 轮，MLP 则延后到第 10 轮。这说明“最合适的停止时刻”并不是一个统一常数，而是取决于模型结构、参数规模和当前优化状态。以 TextCNN 为例，它在第 6 轮就已经达到最优，如果仍然机械地训练到第 10 轮，那么后面的 9 轮更多是在消耗时间；而 MLP 最优轮次更靠后，说明它虽然结构简

单，但也需要更长的迭代来把平均池化后的句向量和分类层磨合到较稳定状态。

固定迭代次数的优点是实现简单、复现实验方便，比如所有模型统一训练 10 轮，实验流程非常直观；但它的缺点也很明显：若轮数设得太小，会让像 MLP 这类收敛稍慢的模型欠拟合；若轮数设得太大，则会让已经达到峰值的模型继续朝训练集拟合，验证集指标反而下降。相比之下，基于验证集的早停更贴近“泛化性能最好时就停止”的目标，只是它需要额外划分验证集，并且要设定 `patience` 这类超参数来容忍短期波动。

如果用一个具体句子来理解这个过程，可以把模型看成在不断学习如何区分“故事拖沓 剪辑混乱 看完只想立刻退场”与“前半段稍慢 后半段反转精彩 整体超出预期”这类表达。训练前几轮，模型通常先学到明显的情感词；继续训练后，可能开始学习更复杂的转折和搭配。但如果再往后训练太久，它也可能把训练集里某些只出现过几次的人名、片名或口头表达当成“捷径特征”，这时训练集指标继续上涨，而验证集 F1 却不再提升，说明应该及时停止。

6.2 实验参数如何初始化

本实验的初始化分成“词向量初始化”和“网络参数初始化”两部分。词向量方面，PAD 向量固定为 0，这样它既不会在平均池化中引入额外噪声，也不会循环状态更新中制造虚假信息；命中预训练词向量的词直接加载已有表示；未命中的词则使用与预训练分布同尺度的随机初始化。这样做的目的，是让模型一开始就拥有基本的语义几何结构，而不是从完全无意义的随机空间重新学起。

例如句子“宣传很热闹 实际内容乏味 完全浪费时间”中，像“宣传”“内容”“时间”这类常见词通常能直接命中预训练词向量，而更少见的口语词或专有词可能需要随机初始化。如果全部词都随机初始化，那么模型在训练初期连“失望”和“推荐”之间的语义差异都还没有建立，收敛会明显变慢；而预训练词向量能让模型从一开始就知道“精彩”“推荐”更接近正向语义，“拖沓”“乏味”更接近负向语义。

网络层方面，线性层权重使用 Xavier 初始化，适合前后层规模接近的全连接变换；卷积层使用 Kaiming 初始化，因为 ReLU 会截断负半轴，Kaiming 更有利于保持激活值方差稳定；RNN、LSTM 和 GRU 的隐藏到隐藏权重使用正交初始化，是因为循环结构会反复把上一时刻状态传给下一时刻，正交矩阵更有助于缓解长序列上的梯度爆炸或梯度消失。偏置统一初始化为 0，避免在训练刚开始时人为给某一类激活增加偏向。

不同初始化方法适合的地方也不同。若把所有权重都初始化成同一个值，例如全部为 0，那么每个神经元接收到的梯度会完全一样，最后学出来的还是同一个特征，网络等价于只有一个神经元；若高斯初始化方差设置过大，前向传播时激活值会一层层放大，训练会很不稳定。相反，像循环层这样需要跨时间传递信息的结构，更强调状态范数不要在 80

个时间步里迅速衰减，因此正交初始化比简单的随机高斯更合适。以“虽然前半段很慢但是后半段非常感人”为例，若循环权重初始化不稳定，模型可能在读到“但是”之后已经忘掉前面的状态变化，难以正确处理转折。

6.3 如何缓解过拟合

过拟合的本质，是模型不仅学到了“什么模式代表正面、什么模式代表负面”，还把训练集中特有但不具备普遍性的细节也一并记住了。对于情感分类任务来说，这些“坏记忆”可能是某个演员名字、某种只在训练集高频出现的口头表达，甚至是几个词偶然同时出现的搭配。其直接表现通常是：训练集损失继续下降，但验证集 F1 不再提升，甚至开始回落。

本实验中最直接的缓解方法有三类。第一类是训练过程控制，例如基于验证集 F1 的早停；当模型在验证集上不再进步时，及时终止训练，避免它继续朝训练集细节过拟合。第二类是显式正则化，例如 MLP 使用 `'dropout=0.3'`，TextCNN 使用 `'dropout=0.5'`，同时训练中还使用 `'weight_decay'` 约束权重过大。dropout 相当于训练时随机“遮住”一部分特征，迫使模型不能只依赖单一词或单一通道；weight decay 则抑制权重无限变大，防止分类边界过于陡峭。

第三类方法是控制模型容量和增加数据多样性。若发现 BiRNN、BiLSTM 这类模型在训练集上很快达到很高分数，但验证集不升反降，可以尝试减小 `'hidden_dim'`、减少层数、减少卷积核数量，或者增加更多训练样本和数据增强。比如 TextCNN 如果把 `'num_filters'` 调得过大，可能会记住训练集中一些只出现过几次的局部短语；而适度的数据增强，如同义词替换、词 dropout，可以让模型看到更多表达变体，不至于把“精彩”这一种写法学成唯一的正面模式。

用例子来说，若训练集中负面影评经常出现“王力宏”“宣传片”“无聊至极”等特定搭配，模型就可能错误地把演员名或某个场景词当成负面信号。一旦测试集中出现“王力宏表现不错”这样的新句子，过拟合严重的模型就容易误判。因此，缓解过拟合不是单纯“让模型变差一点”，而是让它从记忆具体样本，转向学习更可迁移的规律，例如“拖沓”“混乱”“浪费时间”这类更通用的负面证据。

6.4 CNN、RNN 与 MLP 的优缺点

三类模型的核心差异，在于“它们如何把一串词组织成一句话的表示”。MLP 的做法最简单：先把所有有效词向量平均，再交给全连接层分类，因此它计算快、参数少、训练稳定，适合作为 baseline。它的优点是在短句或情感词非常明确时往往已经够用，例如“无聊至极”“十分感人”这类句子，只要平均后的向量里正负词足够明显，MLP 就能做出合理判断。

但 MLP 的问题也很明显，它几乎不关心词序和局部结构。例如“值得推荐”和“不值得推荐”这两句话包含大量相同的词，如果只做平均，模型很难充分体现“不”对后面短语的翻转作用。CNN 的优势就在这里：它会用局部卷积核扫描相邻词，专门捕捉“非常感人”“不值得看”“剧情混乱”这类短语级模式，因此对否定词、程度副词、固定搭配尤其敏感，而且卷积可以并行计算，训练速度通常比循环模型更高。

RNN 及其门控变体的优点，则是显式保留顺序和上下文传递过程。像“虽然前半段很慢但是后半段非常精彩”这样的句子，真正决定极性的往往是后半句以及“但是”所表示的转折关系。普通 RNN 理论上可以逐词更新状态来处理这种结构，但它在长距离依赖上容易遗忘前文；LSTM 通过输入门、遗忘门和输出门维护更强的长期记忆，GRU 则用更轻量的门控实现相近效果，因此它们更适合处理长句、转折句和依赖前后文解释的评价。

代价方面，也必须把训练时间纳入比较，而不能只看准确率。就本实验而言，MLP 训练约 25.4 秒，是五个主模型里最快的；TextCNN 约 166.8 秒，虽然比 MLP 慢，但仍明显快于双向循环模型。双向循环模型的时间成本则更高：BiGRU、BiRNN 和 BiLSTM 分别约为 1893.7、590.9 和 2492.7 秒，大约是 TextCNN 的 11.35、3.54 和 14.94 倍。也就是说，RNN 家族带来的收益并不是“免费”的，尤其 BiLSTM 的训练成本最高，但当前测试集表现最好的是 TextCNN、RNN 家族内部表现最好的是 BiGRU，这说明更复杂的时序建模并不一定会在当前数据规模下自动转化为最高分数。因此从工程折中看，若只追求最快速度，MLP 最轻；若希望在性能与时间之间取得平衡，TextCNN 更合适；若任务更依赖长距离上下文和转折结构，再为 LSTM 或 GRU 付出更高时间成本才更有意义。作为追加补充，Transformer 通过自注意力并行建模全局依赖，但在缺少更强预训练语义和更大训练预算时，优势未必会自然体现出来。

6.5 模型鲁棒性分析

鲁棒性可以从两个层面来理解：一是对“与原训练分布接近”的样本是否还能保持稳定判断，二是当文本领域发生变化时，性能会下降到什么程度。本实验同时做了

这两种测试：一组是手工构造的小规模影评风格句子，另一组是来自微博语体的外部测试集。前者更适合观察模型面对局部短语、转折结构时的反应，后者更适合评估跨域泛化能力。

从手工影评样本看，模型对情感线索非常直接的句子通常反应一致。例如“台词空洞 表演生硬 配乐也非常吵闹”这种句子里，负面线索几乎全部集中在局部短语上，CNN、MLP 和各类 RNN 都比较容易抓住；而像“前半段稍慢 后半段反转精彩 整体超出预期”或“虽然成本不高 但是情感真挚 细节打动人”这样的句子，则更能检验模型是否理解“前弱后强”“虽然……但是……”这类结构。也就是说，鲁棒性不只是看模型会不会分正负，还要看它在不同表达形式下是否仍能抓住真正起决定作用的部分。

表9 小规模影评风格样本上的预测结果对比

样本	真实标签	MLP	TextCNN	BiRNN	BiLSTM	BiGRU	Transformer
剧情紧凑 节奏明快 演员表现很出色 这部电影值得推荐	1	0	0	0	0	0	0
故事拖沓 剪辑混乱 看完只想立刻退场	0	1	1	1	1	1	1
虽然成本不高 但是情感真挚 细节打动人	1	0	0	0	0	0	0
台词空洞 表演生硬 配乐也非常吵闹	0	1	1	1	1	1	1
前半段稍慢 后半段反转精彩 整体超出预期	1	0	0	0	0	0	0
宣传很热闹 实际内容乏味 完全浪费时间	0	1	1	1	1	1	1

为了进一步评估跨域鲁棒性，本实验额外从公开 GitHub 数据集 `weibo2018` 中抽取了 100 条带标签微博文本作为外部测试集，其中正样本 50 条、负样本 50 条。该数据与原任务的中文影评语料存在明显领域差异，包含口语表达、话题标签、表情符号和微博语体，因此能更直接反映模型面对分布偏移时的稳定性。

外部样本分词后共有 4769 个 token，其中只有 54.92% 能在训练词表中命中，显著低于原训练语料的词向量覆盖率。这说明模型在跨域场景下不仅要面对表达习惯变化，还会遭遇更严重的词汇失配问题。

具体结果也印证了这一点。尽管各模型在原测试集上的 F1 大多还能维持在 0.78 到 0.85 左右，但迁移到微博数据后，最佳外部 F1 也只有 0.2051，对应模型为 Transformer；而 F1 降幅最小的模型降幅也达到 0.5845。这说明当前模型对“影评语体内部”的规律掌握得还可以，但一旦遇到口语化更强、缩写更多、情绪表达更跳跃的微博文本，原先学到的模式就会明显失效。简单说，模型学会了如何判断‘剧情拖沓’‘表演生硬’，但不一定学会了如何判断微博里的‘无语住了’‘笑死我了但是真烂’这类表达。

从模型间差异看，Transformer 的外部测试 F1 最好，说明它在当前这组跨域样本上的迁移表现相对更稳；而 Transformer 的 F1 降幅最小，说明它在迁移时退化得稍慢一些。不过整体上看，没有任何模型真正解决跨域问题，因此更合理的结论不是“某个模型已经足够鲁棒”，而是“当前基于单一影评语料训练的模型鲁棒性有限，后续需要靠跨域数据、领域自适应或更强的预训练表示继续补强”。

表10 原测试集与外部微博测试集上的鲁棒性对比

模型	测试集 Acc	测试集 F1	外部集 Acc	外部集 F1	Acc 下降	F1 下降
MLP	0.8428	0.8380	0.3800	0.1143	0.4628	0.7237
TextCNN	0.8564	0.8490	0.3600	0.0588	0.4964	0.7902
BiRNN	0.7805	0.7756	0.3800	0.0312	0.4005	0.7444
BiLSTM	0.8103	0.8056	0.4900	0.1053	0.3203	0.7003
BiGRU	0.8130	0.8150	0.4800	0.0370	0.3330	0.7780
Transformer	0.7805	0.7896	0.3800	0.2051	0.4005	0.5845

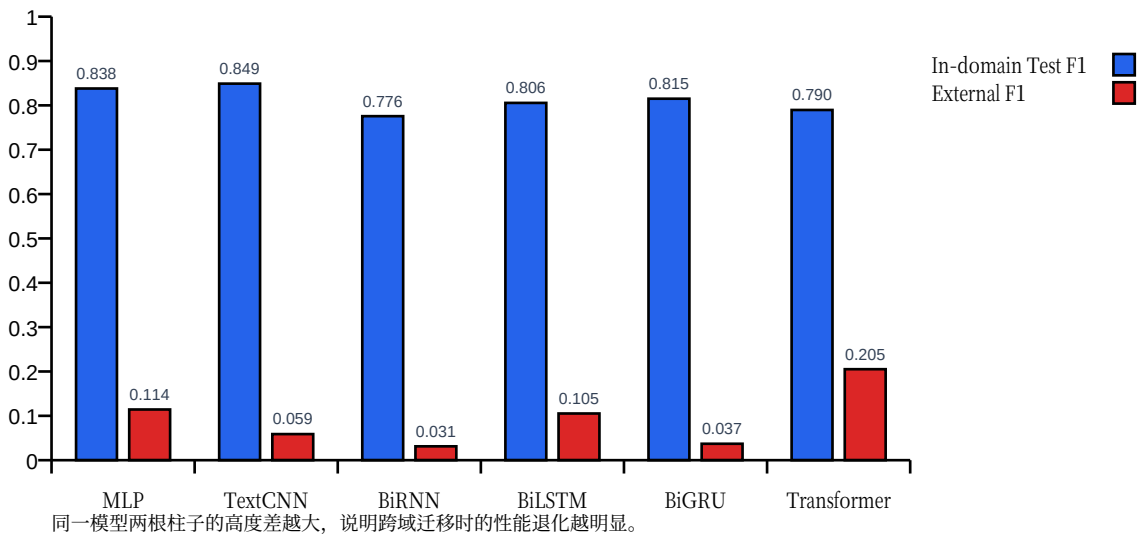


图12 原测试集与外部微博测试集 F1 对比

从对比结果看，参与外部评测的模型在原测试集上的 Accuracy 大约位于 0.78 到 0.86 之间，但迁移到微博数据后，Accuracy 仅剩 0.36 到 0.49，F1 仅剩 0.03 到 0.21；即使下降最小的模型，其 Accuracy 和 F1 也分别下降了 0.3203 与 0.5845。按外部集 Accuracy 看，表现最好的是 BiLSTM；按外部集 F1 看，表现最好的是 Transformer。这说明当前模型更多学习到了影评领域中的局部搭配和表达分布，而不是能够稳定迁移到通用中文情感场景的抽象判别规则，因此其跨域鲁棒性较弱。

7 心得体会

这次实验我前后大约做了两天，也是我第一次严格意义上把 ChatGPT 5.4 的 Codex 当作主要辅助工具来完成一个比较完整的课程实验。最直观的感受就是，AI 的确明显改变了我的学习方式和做实验的节奏。以前我做这类作业，往往会把很多时间耗在环境问题、样板代码、重复性修改和低层级 debug 上；而这一次，在 Codex 的帮助下，我能更快把数据处理、模型搭建、训练脚本、报告生成这些模块串起来，把更多精力放在理解“这一块到底在做什么”以及“下一步应该怎样调参”上。这种效率提升对我来说是非常真实的，也让我第一次很强烈地意识到，AI 已经不只是一个查资料的工具，而是在实际改变我思考问题和组织工作的方式。

但与此同时，我也确实发现了不少问题，而且这些问题不能回避。首先，Codex 在多任务并行、长链路任务保持一致性方面并没有我一开始想象得那么稳定。有些时候我希望它一边整理结果、一边训练新模型、一边更新报告，但它并不能很好地把多个任务都执行得足够扎实。其次，它偶尔会“偷工减料”：例如我明明期望它重新训练某个模型，它却可能直接沿用历史参数、复用旧结果，表面上看流程跑通了，实际上关键实验并没有真正重做。更麻烦的是，训练出来的结果有时还会出现偏差，这时候我很难第一时间判断到底是我自己的指令写得不够清楚、参数设定有问题，还是工具本身出了错。也正因为如此，这次实验让我更深地明白了一点：AI 可以大幅提高效率，但不能替代人工甄别，特别是在实验结果、参数来源和结论解释这些关键环节上，人必须保持清醒，不能因为输出看起来“像对的”就直接接受。

从模型理解上说，这次实验也让我收获很大。以前我对很多模块的认识停留在“知道名字”和“知道大概作用”这个层面，比如为什么 TextCNN 要多卷积核、为什么 RNN 容易在长序列上遗忘、为什么早停和 dropout 会影响最终结果；而这次因为有 AI 帮我更快搭起完整实验链路，我反而有了更多时间回过头去看每个部分到底在干什么。比如在比较 MLP、TextCNN 和 BiRNN/BiLSTM/BiGRU 时，我逐渐意识到，真正重要的不是“哪个模型听起来更高级”，而是它到底以什么方式组织语义：MLP 是平均，CNN 是抓局部模式，RNN 是按顺序传递上下文。这种理解比单纯把代码跑通更重要。某种意义上，这次实验让我第一次觉得，AI 并没有削弱我对模型的理解，反而促使我把注意力从机械排错转向了结构分析和实验判断。

对于具体结果，我一开始的直觉其实是：RNN 路线应该会明显优于 CNN 或 MLP，因为语言本身就是连续的序列，前后文关系、转折结构和长距离依赖看起来都更适合交给循环模型来处理。但真正把实验系统地跑下来以后，我发现不同模型之间的差距并没有想象中那么夸张，很多时候只是“谁更稳一点、谁更均衡一点”，而不是“

谁绝对碾压谁”。这对我来说是一个很有价值的修正：直觉可以提供方向，但不能代替实验。即便如此，我仍然感觉 RNN，特别是带门控的 BiLSTM、BiGRU，在处理连续语言信息时有天然优势，尤其在鲁棒性和上下文保持上通常会更稳一些。也就是说，它不一定在所有指标上都拉开巨大差距，但在面对更复杂表达、转折句、跨域文本时，确实更容易体现出结构上的合理性。

另外，这次实验也让我越来越强烈地感受到：在 AI 时代，“能不能把代码写出来”正在逐渐不再是最难的问题，真正困难的部分反而在于如何优化模型、设计结构、筛选实验、理解结果。超参数很多，结构选择也很多，而我真正要做的不是简单把它们全试一遍，而是尽量用控制变量法把每个超参数调到更合适的位置，并理解它为什么会产生这样的变化。比如这次调参过程中，我明显感觉到效率和准确率有时是矛盾的：更复杂的模型、更大的隐藏维度、更长的训练轮次，未必一定带来更好的最终表现，却几乎一定会带来更高的计算成本和更慢的实验速度。这件事提醒我，做模型不能只盯着“精度最高”这一个目标，还要同时考虑训练成本、收敛速度、可复现性和结构是否真的值得保留。

如果说这次实验里最难、也最让我头疼的部分，我觉得还是调参。真正做起来以后我才发现，调参远比“写出一个能跑的模型”更考验耐心和判断力。因为我必须尽量坚持控制变量法，一次只动一个参数，才能知道性能变化到底是由谁引起的；但这样做也意味着实验数量会迅速膨胀，很多轮训练跑完以后可能只是告诉我“这个方向没有用”。尤其是在多个 epoch 下效果并不理想、甚至后期开始退化时，我会非常直观地感受到过拟合的存在，也更能理解为什么早停不是一个可有可无的小技巧，而是很重要的训练策略。总的来说，这次实验一方面让我真切感受到 AI 工具带来的效率革命，另一方面也让我更加清楚：真正有价值的能力，仍然是提出问题、验证结果、分析误差和做出取舍的能力。也正因为如此，这次实验虽然累，但我做完以后反而有一种很强的兴奋感，因为我感觉自己不仅完成了一次课程作业，也真正体验到了 AI 正在如何重塑我学习和思考的方式。

参考文献

- [1] Kim Y. Convolutional Neural Networks for Sentence Classification[C]//Proceedings of EMNLP 2014. Doha: ACL, 2014: 1746-1751.
- [2] Hochreiter S, Schmidhuber J. Long Short-Term Memory[J]. Neural Computation, 1997, 9(8): 1735-1780.
- [3] Cho K, Van Merriënboer B, Gulcehre C, et al. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation[EB/OL]. arXiv:1406.1078, 2014.
- [4] Mikolov T, Chen K, Corrado G, Dean J. Efficient Estimation of Word Representations in Vector Space[EB/OL]. arXiv:1301.3781, 2013.

[5] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]//Advances in Neural Information Processing Systems 30. 2017.

附录与 AI 的交互记录（节选）

说明：以下内容根据实际开发过程整理，保留了问答式记录的形式，尽量还原我作为初学者一步一步完成程序时的提问方式。

1. 任务理解阶段

我：这次作业到底要写成什么样？我现在只知道是做情感分类，但是不知道程序最后应该有哪些部分。

AI：可以先不要急着写模型，先把整体任务拆开。这个程序至少可以分成几部分：读数据、把文本变成模型输入、定义模型、训练和验证、保存结果、最后生成报告。如果你先把这几个部分想清楚，后面代码就不会乱。

我：也就是说，不是只写一个模型训练脚本就行？

AI：对。你可以把它理解成一个小实验项目，不只是跑一次，而是要能比较不同模型，还要保存结果，最后还能整理成报告。

2. 项目结构阶段

我：我现在最不会的地方其实是代码架构。我以前经常把东西都写在一个 .py 文件里，这次是不是也能这样写？

AI：能写，但是后面会越来越乱。你可以先按功能拆开。比如 data.py 负责数据处理，models.py 负责模型定义，train.py 负责训练和评估，experiment.py 负责实验配置，scripts/ 里放运行入口。这样你后面改某一部分的时候，不会把别的部分弄乱。

我：我对这种拆文件的方式其实不太熟。

AI：没关系，这次可以先按最基础的模块化来做。你不用追求很复杂的设计，只要做到数据处理和模型训练分开，就已经比全部写在一个文件里清楚很多了。

3. 数据读取阶段

我：数据集是一行一条样本，我不知道该怎么处理成模型能用的格式。

AI：可以按最常见的流程来。每一行先拆成标签和分词后的文本，然后建立词表，把每个词变成编号。之后为了方便 batch 训练，再把句子统一成固定长度，太长就截断，太短就补齐。

我：为什么一定要统一长度？

AI：因为神经网络训练时通常是一批一批送进去的，同一批里的输入张量形状最好一致。对初学者来说，先用固定长度是最省事、最稳定的办法。

我：那词表是不是训练集、验证集、测试集都一起建？

AI：更规范的做法是只用训练集建词表，避免信息泄漏。你可以把这点写进报告里，这样会更严谨。

4. 预训练词向量阶段

我：题目要求用预训练词向量，但我现在只知道有一个词向量文件，不知道程序里该怎么接。

AI：最简单的理解是：先把词表里的每个词，去预训练词向量里查对应的向量；然后把这些向量排成一个矩阵；模型开头放一个 embedding 层，用这个矩阵初始化。

我：所以 embedding

层不是我从零开始乱写的，它其实是词编号到词向量的查表过程？

AI：对，可以先这么理解。你不用一开始就纠结很深的理论，先知道程序上它是把词编号映射成向量就够用了。

5. 模型阶段

我：我现在对 MLP、CNN、RNN

这些模型只有大概印象，不知道代码上差别有多大。

AI：可以先抓住共同点。它们的输入都可以是词编号序列，输出都可以是二分类 logits。这样你训练部分就能共用。不同点主要在中间的特征提取方式不一样。

我：那我是不是应该先写一个最简单的模型？

AI：对。你可以先从 MLP 开始，因为它最容易跑通。比如先把一句话的词向量做平均，再接全连接层分类。等这个流程通了，再加 TextCNN 和双向 RNN。

我：这样会不会显得太基础？

AI：对初学者来说，先写基础版本是正常的。重点是先把完整流程做出来，而不是一开始就堆很复杂的结构。

6. 训练流程阶段

我：我现在不太清楚训练循环应该怎么写。是每个模型都单独写一份吗？

AI：不建议。你可以写一个统一的训练函数，比如输入模型、数据和训练配置，输出验证集和测试集结果。这样换模型的时候只需要换模型本身，不需要重写训练逻辑。

我：训练时我应该记录什么？

AI：最基础的有训练损失、验证集准确率、验证集

F1。每一轮训练完就评估一次验证集，然后用验证集 F1 决定当前是不是最好模型。

我：为什么不是看训练集效果？

AI：因为训练集效果好不代表泛化好。保存模型时一般更关心验证集表现，这样更合理。

7. 主实验脚本阶段

我：如果我要比较多个模型，是不是只能手动一个一个改代码运行？

AI：不用。你可以把每个模型的配置整理成一个列表。每个配置里写清楚模型名字、构造方法和训练参数，然后主脚本按顺序遍历它们。

我：这样就是把实验内容写成配置，而不是把逻辑写死在程序里？

AI：对，这是一种很基础但很好用的组织方式。你后面如果再加一个模型，只需要加一条配置，不需要改训练主流程。

8. 超参数理解阶段

我：我有点不好意思，但我现在对超参数这个词其实还不是特别清楚。

AI：这个问题很正常。可以先这样理解：模型参数是训练过程中学出来的，比如权重；超参数是训练开始前你先设定好的，比如学习率、batch size、dropout、hidden_dim 这些。

我：也就是说，超参数不是模型自己学出来的，是我先给它定的？

AI：对。你可以把它理解成训练规则和模型规模的一部分设置。

我：那 learning_rate、batch_size、dropout 分别大概是做什么的？

AI：学习率决定每次更新走多大一步；batch size 决定一次看多少样本；dropout 是一种防止过拟合的方法；hidden_dim 一般表示隐藏层表示的维度大小。你不用一次理解得特别深，先知道它们会影响结果就可以。

9. 调参阶段

我：如果我对超参数还不熟，那调参这部分我应该怎么写比较稳妥？

AI：最适合初学者的方法是控制变量。先固定一个基线模型，然后每次只改一个超参数，其他都不变。这样最后结果变化比较容易解释。

我：比如我先固定 MLP 的其他设置，只改 hidden_dim，再只改 dropout？

AI：对，就是这个思路。

我：这样是不是比同时改很多参数更简单，也更容易写清楚？

AI：是的。虽然未必是最快找到最优结果的方法，但对课程作业和初学者来说，这是最清晰的方法。

10. 调参脚本实现阶段

我：如果我每改一个参数都手写一遍训练命令，感觉会特别麻烦。

AI：那就可以单独写一个调参脚本。先为每个模型设定基线配置，再为每个超参数列出几个候选值，程序自动生成实验并运行。

我：我之前没写过这种脚本，怕自己越写越乱。

AI：可以先从最基础的版本开始。比如只做模型名 + 参数名 + 参数值的实验命名，然后把结果按固定格式保存。只要命名统一，后面汇总和画图都会容易很多。

我：原来调参脚本本质上是在批量组织实验。

AI：对。你可以把它理解成自动帮你重复做很多次单独实验。

11. 结果保存阶段

我：我现在实验越来越多了，最怕的就是结果文件混在一起，后面根本分不清。

AI：那就要早点统一保存规则。比如每个实验保存一个 metrics.json，最优模型保存一个 best.pt，最后再生成一个总的 summary.json。

我：这样后面画图就不用重新到处翻结果了？

AI：对。你后面不管是做表格、画曲线还是写报告，直接从汇总文件读取会轻松很多。

12. 报告生成阶段

我：我原来以为图表都要手工画，现在感觉数据很多，手工整理会很容易出错。

AI：可以直接写程序生成图。比如训练历史可以画 loss 曲线和验证集 F1 曲线，多个模型结果可以画柱状图，调参结果可以单独画每个参数的对比图。

我：这样是不是也更像一个完整项目，而不是只会跑训练？

AI：对，而且自动生成图表还有一个好处，就是如果你后面重新跑了实验，只需要重新执行脚本，图和表也能一起更新。

13. 开发过程中的整体感受

我：我现在回头看，感觉如果没有这种一步一步问，我自己一开始根本不知道从哪里下手。

AI：这很正常。对初学者来说，最难的往往不是某一行代码，而是不知道整体顺序。只要先把问题拆成读数据、写模型、训练、保存结果、做报告这几个步骤，后面就会清楚很多。

我：我觉得这次我最大的变化不是一下子学会了很多理论，而是慢慢知道了一个程序应该怎么组织。

AI：这其实很重要。课程作业除了结果本身，代码结构、实验过程和结果整理能力也都很关键。

14. 最后整理阶段

我：如果我要把这次和 AI 的交互写进附录，是不是不用写得特别像专家？

AI：对，最好保留你真实的学习过程。比如一开始不懂代码架构、不清楚超参数是什么意思、不知道怎么做调参，这些都可以写进去。这样反而更真实，也更符合初学者逐步完成程序的过程。

我：明白了。那我就按我先问最基础的问题，再一点点把程序搭起来的思路整理。

AI：这样写是合适的，也更符合你这次开发的实际过程。